VIBE CODING 教學案例 01

器 VESSEL
陶藝工作室一頁式網站

把「我想要這樣」變成 AI 能穩定執行的規則。
網站只做一次，這套流程之後每個專案都用得上。

成品　台北大稻埕一家柴燒陶藝工作室「器 VESSEL」的一頁式品牌官網。深色底，火焰橘只出現在溫度與游標焦點，沒有購物車，也沒有「服務項目」這類泛用區塊。目的是讓看到的人願意親自去工作室走一趟。

使用技術　Astro · TypeScript · 原生 CSS · Vercel

時間　閱讀約 1.9 小時，自己動手約 5.7 小時

對象　沒有程式背景，全程不需要自己寫程式碼

完成等級　Functional Prototype（功能完整的原型）

核心　AI 的品質取決於你有沒有把需求、限制與驗收標準說清楚

案例一：器 VESSEL

把「我想要這樣」變成 AI 能穩定執行的規則

> 這份手冊是寫給沒有程式背景的人的。
>
> 你不需要會寫 HTML、CSS 或 JavaScript。從頭到尾，你不會自己寫任何一行程式碼。
>
> 這份手冊的重點是**判斷**：這個設計對不對、AI 給的東西該不該收、以及怎麼把「我喜歡這樣」變成 AI 做得出來的規則。

這個案例要回答的問題

> 我要如何把「我想要這樣」，變成 AI 可以穩定執行的規則？

這個案例做的是一個陶藝工作室網站。階段的順序，換一個專案還是同一套。

目錄

開始之前

- 學員路線圖
- 這個案例做到最後，完成的是什麼？
- 這個專案是怎麼被建起來的
- 全課地圖
- 開始之前：環境準備
  - 你需要準備的東西
  - 這個案例你要親手做什麼
  - 環境檢查（五分鐘）
  - 認識 Claude Code Desktop 的畫面
  - 安裝這個案例會用到的 Skill
  - 接上 GitHub（階段 12 才會用到）
- 這份手冊怎麼用

隨時可以翻回來查的

- 四張操作卡
  - 操作卡 1 和 AI 一起做事的七個步驟
  - 操作卡 2 哪些事 AI 可以自己做，哪些要先問你
  - 操作卡 3 一個能用的指令，有六個構件
  - 操作卡 4 卡住的時候怎麼辦
- 不會寫程式，怎麼審查 AI 的產出

階段

1. 先看成品，找出「AI 感」是哪些東西
2. 先讓 AI 提計畫，你確認後才動手
3. 建立專案，看懂 AI 要執行什麼
   - Checkpoint 1（做完階段 3）
4. 把參考圖變成一份視覺規則
5. 把視覺規則寫成 AI 每次都會讀的設計規範
6. 把顏色與尺寸集中成一份設定
   - Checkpoint 2（做完階段 6）
7. 做出第一個畫面，並驗收它有沒有照規範
8. 加上自動檢查，讓違反規範的改動被退回
9. 在自動檢查的保護下完成其餘區塊
   - Checkpoint 3（做完階段 9）
10. 量四個尺寸，找出眼睛看不出來的問題
11. 讓 AI 審查 AI，並學會拒絕它的建議
12. 推上 GitHub 並部署成公開網址

做完之後

- 完成清單
- 離開這個範例，我會不會自己做？
- 常見卡關
- 名詞白話對照表
- 各階段預估時間

🗺️ 學員路線圖

起點	桌面上一個空資料夾，裡面只有你自己寫的 `spec.md`
過程	從參考圖談出視覺規則。寫成 `CLAUDE.md` 的設計規範與 `tokens.css`。加上一個會擋下違規改動的自動檢查。檢查生效之後，才做其餘區塊
產出	一個公開網址上的一頁式品牌網站，以及一份機器檢查得了的設計規範
核心判斷	這條規則寫得夠不夠具體：能不能被機器檢查，或者被人眼一秒判斷。階段 5 寫下，階段 8 檢驗

其餘的判斷、名詞、指令都可以邊做邊查。**只有最後那一列，換一個專案還是同一套。**

上面那張表的〈過程〉出現了四個名詞。先各看一行就好。

> **名詞：`CLAUDE.md`** 是放在專案資料夾裡的一份說明書，你每開一次新對話，Claude 都會先讀它。
> 你需要知道到什麼程度：知道規則寫在這裡才會一直生效，寫在對話裡講完就忘。階段 5 會實際寫一份。
>
> **名詞：設計規範**是 `CLAUDE.md` 裡專門管視覺的那一段：顏色、圓角、字體、哪些東西不准出現。
> 你需要知道到什麼程度：知道它管的是文字規則，數字的部分交給 `tokens.css`。
>
> **名詞：Design Token**（放在 `tokens.css` 裡）是集中定義的顏色、間距、圓角數字。全站都引用它。
> 你需要知道到什麼程度：知道全站只有這一個檔案可以出現色碼。階段 6 會建。
>
> **名詞：自動檢查**（hook）是 Claude 每改完一個檔案就自動跑一次的規則檢查。
> 你需要知道到什麼程度：知道它由工具執行，不是由 Claude 執行。所以 Claude 沒辦法忽略它。階段 8 會裝。

> **關於「階段」**：階段是這份手冊的章節單位。這個案例共 12 個（階段 1 → 階段 12），照順序做。

這個案例做到最後，完成的是什麼？

> 目前等級 ▓▓▓

要升到 Production-ready Baseline（可以正式上線的基本版本），還差三件事

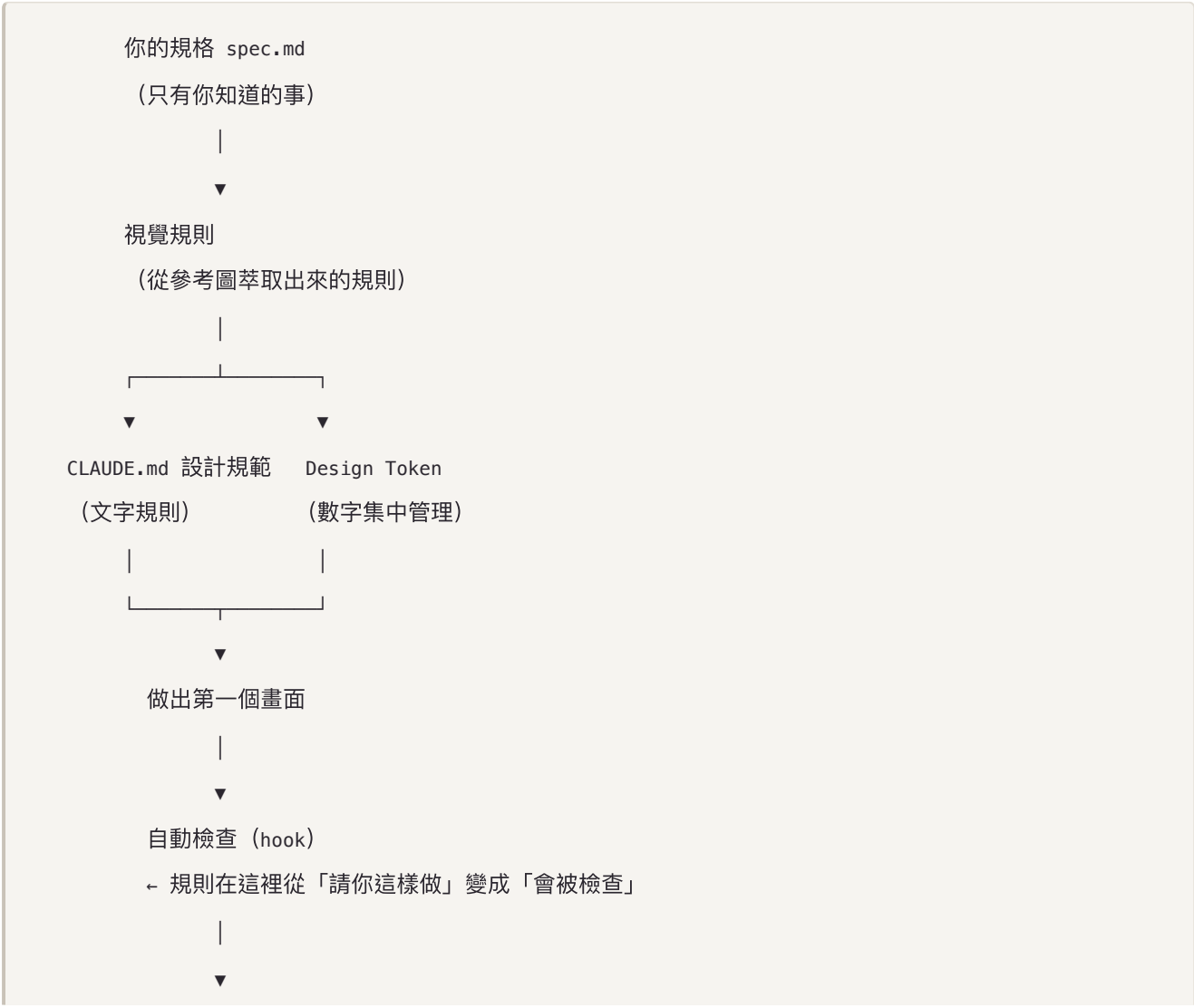
1. 照片改成自己拍的，並且放進自己的專案。現在用的是 Unsplash 的圖庫連結，對方的服務沒有穩定度保證
2. 字體改成自己託管（現在用 Google Fonts，同樣是外部服務）
3. 補上搜尋引擎與社群分享需要的設定（頁面標題、描述、分享縮圖）

第 3 件參考成品已經補上，在 `index.html` 開頭的 `og:` 那幾行和網址列小圖示（favicon）。你的版本可以照著抄。 `og:` 開頭的那幾行，決定網址貼到 LINE、Facebook 時顯示的標題、描述和縮圖。

這三件事都不難，但**這個案例不做**，因為它們跟這裡要教的東西無關。教材不會宣稱你做完就有一個可以正式上線的網站。你手上的東西目前在哪個等級、還差哪三件事，你要自己說得出來。

這個專案是怎麼被建起來的

真正要記住的是**這張圖的順序**，不是階段編號。



```
        完成其餘區塊
      （在檢查的保護下產出）
              |
              ▼
體檢 → 審查 → 上線
```

為什麼不能反過來

順序	如果反過來會怎樣
規格 → 視覺規則	沒有規格就先看參考圖，你會照抄整張圖。你其實不想要的部分也一起抄走
視覺規則 → 設計規範	規則沒談定就先寫進 `CLAUDE.md`。寫出來的會是「要有質感」這種沒有作用的句子
設計規範 → tokens → 第一個畫面	顏色沒有集中管理就先做畫面。色碼會散落在十幾個檔案裡，之後改配色改不乾淨
第一個畫面 → 自動檢查	沒有先做一個畫面出來，你不知道要檢查什麼。檢查規則會變成憑空想像
自動檢查 → 其餘區塊	先裝檢查，後面的區塊才會被保護。反過來的話，你要自己一個一個看
完成 → 體檢 → 上線	沒做完就體檢，會一直重複檢查同樣的問題

你之後做自己的專案，用的也是這個順序。

全課地圖

階段	產出	重點
1	一份「AI 感」清單，補進 `spec.md`	打開兩個版本並排，先自己寫下差別，再對照參考答案
2	AI 的一份計畫（還沒有任何檔案）	切到 Plan 模式，貼上第一個指令
3	一個空的專案結構	看懂建立專案的指令，再按允許
4	一組顏色 + 首頁大綱 `outline.md`	拖參考圖進對話框，再貼 `spec.md`
5	`CLAUDE.md` 裡的設計規範	跑 `/init`，把設計規範追加進去
6	`tokens.css`	集中色碼與間距到 `tokens.css`
7	首頁的第一個畫面	描述第一個畫面的版面與照片處理

階段	產出	重點
8	一個會擋下違規改動的自動檢查	寫檢查腳本，掛成改檔後自動跑
9	完整的首頁	做完一個區塊就看一次，再往下
10	四個尺寸的實測數據	量四個寬度，逐項決定修不修
11	一份分成三類的問題清單	開新對話跑 `/code-review`，逐條裁決
12	一個公開的網址	確認 Claude 列出的指令，讓它建專案並推上去

「產出」那一欄只用在這個專案。「重點」那一欄的動作，換一個專案還是同一套。

> **名詞**：**部署**是把做好的網站送上網路，讓別人用一個網址打得開。**Vercel** 是提供這件事的服務。
> 你需要知道到什麼程度：知道 Vercel 只要接一次 GitHub。之後你每次推上去，它就自己重新部署。
> 階段 12 會接一次。
>
> **名詞**：`/code-review` 是 Claude Code 內建的指令。它會掃過專案，列出它認為有問題的地方。
> 你需要知道到什麼程度：知道它列的是通用軟體工程的意見。和設計規範衝突時，以規範為準。階段 11 會用。

開始之前：環境準備

你需要準備的東西

項目	說明
Claude Code Desktop	桌面版應用程式，到 claude.com/download 下載安裝，用你的 Claude 帳號登入。整個案例都在裡面完成，你不需要打開終端機
一個付費的 Claude 方案	**免費方案打不開 Code 分頁。** 需要 Pro、Max、Team 或 Enterprise 其中一種。上課前先確認
Windows 使用者多一步	第一次打開 Code 分頁之前要先裝 Git for Windows，裝完把應用程式關掉重開。macOS 不用
Node.js 22.12 以上	下面會教怎麼確認與安裝
一個 GitHub 帳號	階段 12 上線時會用到。先去 github.com 註冊好，再照下面〈接上 GitHub〉登入一次。Vercel 在階段 12 用這個帳號登入，不用另外準備
你的 `spec.md`	課前作業，見〈規格文件怎麼寫〉（`case-01-spec-writing.pdf`）
三到五張參考圖	你喜歡的網站截圖，存在 `vessel/ref/`（課前作業的做法）。階段 3、4 會用到
課程附件	到課程頁面下載 `case-01-vessel.zip`，解壓縮到桌面。階段 1 會用

這個案例你要親手做什麼

動作	難度
複製手冊上的 Prompt，貼給 Claude	不難，這是主要工作
決定要不要允許 Claude 執行某個動作	不難，它會跳出卡片問你，按一下就好
判斷產出的畫面對不對	**這是這個案例的重點**
自己打指令	不用。指令都由 Claude 執行
看懂 AI 寫出來的程式碼	不用

手冊裡還是會出現一些指令（像是 `npm run build`）。它們的作用是**讓你看懂 Claude 在做什麼**，不是要你自己打。每一個都會說明它在幹嘛。

環境檢查（五分鐘）

先做完這一步再開始。 環境沒弄好就往下走，後面每一步都會出錯，而且錯誤訊息看起來跟環境無關。

動手

打開 Claude Code Desktop，按左上角的 **Code** 分頁，開一個新對話。**Project folder** 隨便選一個空資料夾就好。它在輸入框上方那一排，位置見下一節的定位圖。把這段貼進去：

貼給 AI

你只需要修改：沒有。整段照貼，四個問題一個都不要刪。

```
幫我檢查這台電腦能不能做網站開發。請逐項回報，先不要幫我安裝任何東西：

1. Node.js 有沒有裝？版本是多少？（我需要 22.12 以上）
2. npm 能不能正常執行？
3. 我對目前這個資料夾有沒有寫入權限？
4. 能不能連到 npm 的伺服器？（隨便查一個套件的版本就好）

每一項都用「可以」或「不行」開頭。
不行的話告訴我：問題是什麼、對一個沒有程式背景的人來說最簡單的解法是什麼。
```

四項都「可以」就準備好了。

常見狀況

訊息	意思	怎麼辦
`command not found: npm`	這台電腦沒有裝 Node.js	到 nodejs.org 下載 **LTS** 版本（長期支援版，比較穩），一路按下一步
版本是 18、20 這種數字	太舊了	同上，重裝一次 LTS 版本
Windows 上 Code 分頁打不開	少了 Git for Windows	到 git-scm.com 安裝，裝完把應用程式重開
`EACCES` 或 `permission denied`	那個資料夾不讓你寫東西	換一個資料夾，例如在桌面上新建一個
安裝一直卡住不動	網路擋住了 npm	換一個網路（手機熱點通常可以）。公司電腦的話問一下 IT

訊息	意思	怎麼辦
Views 選單裡沒有 Browser 面板	應用程式版本太舊	選單列的 **Claude → Check for Updates**（Windows 是 **Help → Check for Updates**）
硬碟空間不足	一個專案大約要 200–300MB	清一些空間再來

> **名詞**：**Node.js** 是「在你電腦上執行 JavaScript」的工具。你的網站要先在電腦上組裝起來，組裝就是它在做。
> 你需要知道到什麼程度：知道你不會直接用到它，但少了它什麼都跑不動。

認識 Claude Code Desktop 的畫面

打開應用程式之後，**視窗左上角有 Home 和 Code 兩個分頁。這個案例只會用到 Code 分頁。**

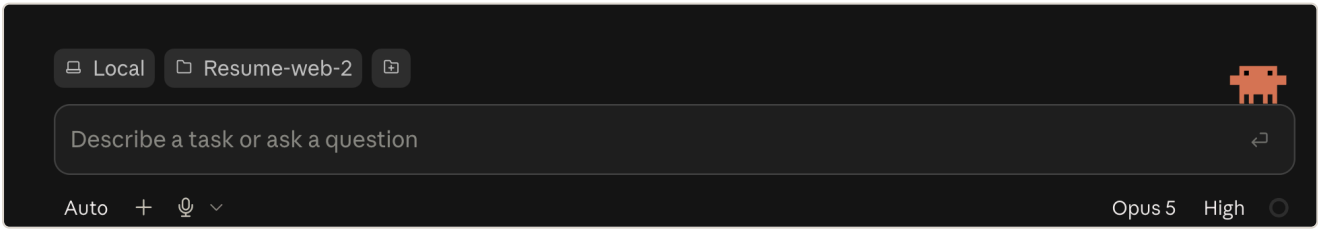
（`Chat` 和 `Cowork` 不是分頁。它們是 Home 畫面輸入框右下角的兩個切換鈕，這個案例不會用到。）

🖥️ 介面定位圖：只會碰到六個地方

這個案例只會用到下面六個位置。找不到某一項時回來看這張表。



> **你要看的是什麼**：① 在**視窗最左上角**，和 Home 並排。打開應用程式時會停在 **Home**，**每次都要先按一下 Code**，這份手冊講的所有東西才會出現在你眼前。



> **你要看的是什麼**：② ③ ④ 三個設定全部集中在**視窗最下方這一條**，不在任何選單裡。左上角的 `Local` 是 Environment。旁邊的資料夾名稱（圖上是 `Resume-web-2`）是 Project folder，再旁邊那個小按鈕可以換資料夾。輸入框**下面**那一排的最左邊（圖上是 `Auto`）就是權限模式選單。大部分時間要把它切成 **Plan** 或 **Manual**。右下角的 `Opus 5` 是模型，用預設的就好。

#	名稱	在哪裡找	用途
①	**Code**（分頁）	視窗**左上角**，和 Home 並排（見上面第一張圖）	整門課只待在這個分頁。開啟時停在 Home，要先按 Code
②	**Environment**	輸入框**上方**那一排的第一個（上圖是 `Local`）	選 **Local**，檔案才會建在你自己的電腦上
③	**Project folder**	緊接在 Environment 右邊（上圖是 `Resume-web-2`）	指定 Claude 只能在哪個資料夾裡工作
④	**模式選單**（Plan / Manual ...）	輸入框**下方**那一排的最左邊（上圖是 `Auto`），隨時可切	階段 2、4 先用 **Plan** 看計畫，確認後切 **Manual**。階段 6–10 直接用 **Manual**
⑤	**Views → Browser**	選單列的 **Views**（上圖沒拍到）	直接預覽你做的網站，可以拖到對話框旁邊並排
⑥	**Views → Diff**	同一個 Views 選單（上圖沒拍到）	看 Claude 這次改了哪幾行。**這是你最主要的驗收工具**

開始一個對話之前，先設定三件事

位置都見上面的定位圖，這裡只講要選什麼：

1. **Environment** 選 `Local`
2. **Project folder** 選你的 `vessel` 資料夾
3. **模式選單** 見下一段

模型用預設的就好。

權限模式：決定 AI 什麼時候要先問你

模式選單在輸入框**下方**那一排的最左邊（見上面的介面定位圖），隨時可以切換。

模式	白話	用在哪
Plan	只規劃，不動手	階段 2、4 用它，先看 Claude 打算怎麼做
Manual	每一個改動都先問你	確認計畫之後執行用這個。階段 6–10 全程用它
Accept edits	自動接受改檔，但執行指令還是會問	熟了之後可以換成這個，比較快
Auto	自動執行，背後有安全檢查	這個案例用不到
Bypass permissions	什麼都不問	不要用

> **Plan 模式只套用在目前這個對話**，開新對話要重選。其他模式會記住你在這個資料夾的選擇。

> 舊版本的介面把這幾個模式叫做 Ask permissions、Auto accept edits、Plan mode。名字不同，意思一樣。

三個好用的面板

從 **Views** 選單打開，可以拖到旁邊並排著看：

- **Diff**：Claude 這次改了哪幾行，一行一行對照。**這是你最主要的驗收工具**
- **Browser**：直接預覽你做的網站，不用另外開瀏覽器
- **Terminal**：如果你想自己下指令。這個案例用不到

Claude 動手之前會問你

在 Manual 模式下，Claude 每次要改檔案或執行指令都會停下來。它會跳出一張卡片等你決定。

看不懂那張卡片在說什麼的時候，不要按允許。 直接在對話框問它：「你剛剛那個動作是要做什麼？為什麼需要？」

安裝這個案例會用到的 Skill

什麼是 Skill

Skill 是一包預先寫好的專業指示。裝了之後，你可以用 `/名字` 直接叫出來，Claude 就會照那一包指示做事。

沒裝的時候 Claude 還是會做，只是憑一般常識做。裝了之後，它會照那一包指示裡的專業做法做。

這個案例會用到五個，其中兩個要另外裝

指令	階段	安裝
`/init`	階段 5	內建，直接可用
`/hooks`	階段 8	內建，直接可用
`/code-review`	階段 11	內建，直接可用
`/frontend-design`	階段 7	**要裝**（進階加強 / 非必要）

指令	階段	安裝
`/impeccable`	階段 10	**要裝**（進階加強 / 非必要）

> **要另外裝的那兩個是進階加強，不是必要的。** `/frontend-design` 和 `/impeccable` 裝不起來（公司電腦限制、網路擋住、指令找不到） **完全不影響你完成這個案例**。階段 7 和階段 10 都附了替代寫法，效果會差一點，但十二個階段一步都不會少。第一次上課建議先跳過安裝，整個流程走完一次之後再回來裝。

怎麼裝 `/frontend-design`（進階加強 / 非必要）

它在 Anthropic 的官方外掛集合裡。在對話框輸入這兩行，一行一行來：

```
/plugin marketplace add anthropics/claude-plugins-official
```

```
/plugin install frontend-design@claude-plugins-official
```

> 也可以用滑鼠：輸入框下方那排、模式選單右邊的 **+** 按鈕裡有 plugins 的入口。

怎麼裝 `/impeccable`（進階加強 / 非必要）

它是另一個團隊做的設計工具（impeccable.style），用 npm 發布。把這句貼給 Claude：

貼給 AI

```
幫我安裝 impeccable 這個設計 skill，指令是 npx impeccable skills install。
裝的時候選 claude 這個 provider。
裝完之後告訴我怎麼確認裝好了。
```

裝完之後在對話框輸入 `/impeccable init` 做一次初始化。

怎麼確認裝好了

在對話框輸入一個 `/`，會跳出可用指令的清單。清單裡看得到 `frontend-design` 和 `impeccable` 就成功了。

看不到的話，重開一次應用程式再看。

裝不起來怎麼辦

完全不影響你完成這個案例。 安裝失敗就直接往下做，階段 7 和階段 10 都附了「沒有這個指令時要怎麼做」的替代作法。

接上 GitHub（階段 12 才會用到）

階段 12 要讓 Claude 建一個 GitHub 專案，再把程式碼推上去。它用的是 GitHub 官方的指令列工具 `gh`。這一節把它裝好、登入一次。之後不用再做。

> **名詞：`gh`** 是 GitHub 官方的指令列工具。Claude 用它建專案、推程式碼，不用開網頁。
> 你需要知道到什麼程度：知道它要先登入一次。登入在瀏覽器裡完成，密碼不會經過 Claude。

📋 貼給 AI（裝 gh 並登入）

貼給 AI

你只需要修改：沒有。整段照貼。**第 1 點「我同意之後再裝」不要刪。**

```
幫我安裝 GitHub 的指令列工具 gh，然後登入我的 GitHub 帳號。

1. 先檢查 gh 是不是已經裝了。沒有的話，告訴我你要用什麼方式安裝、
   要執行什麼指令，我同意之後再裝
2. 登入用瀏覽器的方式：gh auth login --web --git-protocol https --hostname github.com
   它會印出一組一次性代碼和一個網址，請把這兩個原樣給我看
3. 我在瀏覽器完成之後，用 gh auth status 確認登入成功
```

你自己做（在瀏覽器完成登入）

1. 打開 Claude 給你的網址（`https://github.com/login/device`）
2. 用你的 GitHub 帳號登入，貼上那組代碼，照畫面指示授權
3. 回到 Claude Code，跟它說「好了」

> **它要你輸入密碼或 token 怎麼辦**：不要在對話框裡輸入。回它「請改用瀏覽器登入的方式」。

✅ 怎麼知道做對了

- [ ] `gh auth status` 顯示已經登入 github.com，後面是你的帳號名稱

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
Mac 上 Claude 說沒有 Homebrew	這台電腦沒有 Homebrew 這個安裝工具	到 cli.github.com 下載安裝檔自己裝，裝完回來從第 2 點繼續
瀏覽器說代碼無效	代碼過期了，或用了上一次的代碼	「代碼過期了，請重新執行一次登入」
Windows 或公司電腦裝不了 `gh`	安裝可能需要 IT 或系統管理員權限	「請說明用 winget 安裝 gh 的方式；如果裝不了，教我改用 GitHub 網站手動建立專案」

這一步失敗可以跳過嗎：可以，只影響階段 12。

四張操作卡

這四張卡不是這個案例的內容，是和 AI 一起做事的通用方法。 每個案例裡都一樣，你之後做自己的專案也用得上。建議印出來，或者截圖存在手機裡。

不用現在背下來。先掃過一遍知道有這四張，遇到下面這些情況再翻回來：

情況	操作卡
要開始一件新的工作，不知道從哪一步下手	**操作卡 1**：七個步驟的順序
Claude 說它要做某件事，你不確定該不該讓它做	**操作卡 2**：哪些可以放手、哪些要你先點頭
想不出指令怎麼寫，或者寫了但它答非所問	**操作卡 3**：一個能用的指令有哪六塊
卡住超過 20 分鐘	**操作卡 4**：卡住的三步處理法
做到後面，Claude 開始違反前面講好的規則	**操作卡 4**：最後一節〈對話太長的時候怎麼辦〉

手冊各階段的 🆘 只處理那一步特有的問題。**手冊沒寫到的情況，回來看這四張。**

操作卡 1｜和 AI 一起做事的七個步驟

```
Step 1  先把需求講清楚
        要做什麼 / 不做什麼 / 有什麼限制 / 怎麼算完成

Step 2  新的、大的改動，先讓 AI 規劃，不要直接改
        什麼時候切到 Plan：
          要做新功能、要大範圍修改、規格還沒談定
        切到 Plan 模式，並明講：
        「先不要修改任何檔案。請告訴我你準備怎麼做、
          會改哪些檔案、每一步的目的，以及可能的風險。」
        手冊寫「這一步用 Manual」的階段，
        是照已經談定的規格或計畫做，直接跳到 Step 4

Step 3  看過計畫，再切回 Manual 讓它執行
        看三件事：有沒有誤解需求 / 有沒有多做不需要的事 /
        有沒有要碰不該碰的檔案或外部資源
        確認完一定要切回 Manual，Plan 模式下它不會動任何檔案
```

```
Step 4  一次只做一小段
        執行 → 看結果 → 驗收 → 確認 → 再往下一步
        不要一次叫 AI 重做整個專案

Step 5  照手冊的驗收條件檢查
        不要用「好像可以」「看起來差不多」
        要用看得到的結果、量得到的數字、跑得出來的測試
        不知道該看哪裡、該怎麼量，就把這一句貼過去：
        「請列出這次改動裡，我在畫面上看不出來、
          但會出錯的地方，以及我要怎麼試。」

Step 6  失敗時不要猜
        把錯誤訊息整段貼給 Claude，再加一句：
        「請先不要修改任何東西。先告訴我最可能的三個原因，
          以及你準備先檢查哪一個。」

Step 7  修完之後重新驗收一次
        「沒有錯誤訊息」不等於「做對了」
```

操作卡 2｜哪些事 AI 可以自己做，哪些要先問你

層級	內容
AI 可以自己做	讀取專案檔案 / 修改本機檔案 / 執行 build（打包） / 執行測試 / 執行檢查 / 分析錯誤 / 建立本機的存檔紀錄（commit）
執行前要你確認	安裝新的工具 / 改動電腦的全域設定 / 大量刪除檔案 / 存取專案資料夾以外的地方
一定要你點頭，它才能做	建立 GitHub 專案 / 第一次把程式碼推上去 / 建立雲端資源 / 部署到公開網址 / 修改正式資料 / 付費 / 發送訊息 / 刪除遠端資源

第三層的原則是：**對外的、要花錢的、不可逆的，AI 不能自己決定。**

重點不是「一定要你親手按」，而是**它動手之前你必須先看到它要做什麼**。這幾件事做下去就收不回來，你要有機會在執行前喊停。正確的說法是：

> 請你先準備好需要的內容與指令，把即將進行的對外操作說明給我聽，我確認之後你再執行。

操作卡 3｜一個能用的指令，有六個構件

手冊上的每一個 Prompt 都是照這六個構件組出來的。**要寫自己的指令時，照同一個順序填。**

#	構件	寫什麼
1	要做什麼	一句話講完
2	不要做什麼	負面清單。這一項最常被漏掉，也最有效
3	限制是什麼	技術、數值、要遵守哪份文件
4	先規劃還是先動手	「先不要改任何檔案，先告訴我你打算怎麼做」
5	怎麼算完成	看得到、量得到、跑得出來的條件
6	哪裡要停下來問我	對外的、要花錢的、要裝東西的

手冊上的 Prompt 怎麼改成你自己的

- `〈…〉` 裡的內容**一定要換掉**
- 其餘的可以照你的專案改寫，但**六個構件盡量都留著**
- **不確定哪裡能動**：手冊上每一段可以貼的 Prompt，上面都有一行「**你只需要修改**：…」。照那一行做就好，沒被點到的部分整段照貼，不用看懂

三個馬上可以用的句型

貼給 AI

先不要寫程式碼。用沒有程式背景的人聽得懂的方式，
告訴我你打算怎麼做、分成哪幾步、每一步在解決什麼問題。
我確認之後你再寫。

貼給 AI

只做 **〈這一段〉**，其他的等一下再說。

貼給 AI

有錯先告訴我是什麼，不要直接改。

操作卡 4｜卡住的時候怎麼辦

依序做這三步

第 1 步　照那個階段的「🆘 如果沒有成功」做一次

第 2 步　把下面的 Recovery Prompt 貼給 Claude，一次只處理一個原因

第 3 步　還是不行 → 先跳過

每個階段的 🆘 都會註明「這一步失敗可以跳過嗎」

卡超過 20 分鐘還沒解決，繼續卡下去沒有意義

Recovery Prompt

貼給 AI

我照著教材做到這一步，但沒有得到預期的結果。

我沒有程式背景。

教材說我應該看到：

〈貼上手冊裡那一階段的驗收條件〉

我實際看到的是：

〈描述畫面，或整段貼上錯誤訊息〉

請先不要修改任何東西。先告訴我：

1. 最可能是哪三個原因
2. 你準備先檢查哪一個

3. 為什麼是這一個
4. 你需要我提供什麼資訊

一次只處理一個原因。

「整段貼上錯誤訊息」不要偷懶。 真正有用的線索，常常不在最紅、最顯眼的那一行。它在上面幾行不起眼的路徑或版本號裡。

「先不要修改任何東西」也是刻意的。 讓 AI 自己想辦法繞過去，它可能會裝一堆你不知道的東西，或是改掉某個設定。問題會藏得更深。先知道原因，再讓它改。

對話太長的時候怎麼辦

症狀：做到後面，Claude 開始違反前面講好的規則，或是重做已經做過的事。對話越長，前面講過的內容在它眼裡佔的份量越低。

情況	做法
同一件工作還沒做完，對話已經很長	在對話框輸入 `/compact`。它會把前面的對話壓縮成摘要，接著做
這一件做完了，要開始下一件	先確認談好的規則都寫進 `CLAUDE.md`，再輸入 `/clear` 或開一個新對話

`/clear` 之後，Claude 只看得到 `CLAUDE.md` 和專案裡的檔案。談好但沒寫進檔案的決定，換了對話就不見了。接著上一段的 Prompt，例如「照你剛剛列的做」，要和前一段在同一個對話裡做。中間不要 `/clear`。換對話之前先問一句：

貼給 AI

我要開一個新對話繼續做。

這個對話裡談定、但還沒寫進 CLAUDE.md 或任何檔案的決定，請列出來。

我確認之後，請把需要保留的寫進 CLAUDE.md。

不會寫程式，怎麼審查 AI 的產出

你不需要讀懂每一行程式碼。你需要的是六個不用讀程式碼也能做的檢查。

一、先要它用白話解釋，再讓它寫

等它整段寫完才發現方向錯了，重來一次的成本高得多。在任何一個 Prompt 的最後加上這一段，可以攔掉大部分的誤解：

貼給 AI

> 先不要寫程式碼。用沒有程式背景的人聽得懂的方式，
> 告訴我你打算怎麼做、分成哪幾步、每一步在解決什麼問題。
> 我確認之後你再寫。

二、用眼睛驗收，不用程式碼驗收

這個案例的每一個 ✅ 驗收，都是看得到的東西：

- 拿掉 logo，還認得出這是陶藝工作室嗎？
- 把照片和文案換成咖啡店，版型還成立嗎？
- 縮到手機寬度，畫面會不會左右滑動？

這些都不需要打開程式碼就能判斷。這個案例的驗收都這樣設計，所以不看程式碼也驗收得了。

三、把規則交給機器檢查

你看不出來的東西，交給階段 8 的自動檢查。它會在 Claude 每次改完檔案後跑一遍規則。違反了就要求它修正，不需要你盯著。

> 階段 5 要求「規則不能寫成形容詞」，原因就在這裡。「要有質感」機器檢查不了，「圓角上限 8px」可以。

四、問它自己最可能錯在哪

貼給 AI

這次的改動如果會出問題，最可能是哪裡？

請列出三個，並告訴我各自的症狀長什麼樣。

它列出的那三個「症狀」要留著。畫面怪怪的時候，對照那份清單就知道要問什麼。

五、確認它只改了該改的東西

貼給 AI

你這次改了哪些檔案？每一個各改了什麼、為什麼？

AI 有時會順手改掉別的東西。這一問可以及早發現。

六、紅旗清單：看到這些就停下來問

這些字會出現在 **Diff** 面板的新增行裡，不用讀懂整段程式碼，掃過去有沒有就好。

看到這個	為什麼要警覺
密碼、password、金鑰被寫死在程式碼裡	這些不該直接寫在檔案裡
`!important` 出現很多次	通常代表樣式在互相蓋來蓋去，設計規範已經被繞過了
同一個顏色值被寫死在很多地方	應該引用 Design Token，否則之後改不動
`border-radius` 出現大於 8px 的值	違反這個專案的設計規範
自動檢查（hook）被關掉或被跳過	有人為了讓程式碼通過而把檢查拿掉了

看到上面任何一項，就把那一行貼給 Claude 問：

貼給 AI

這一行是做什麼的？有沒有違反我的設計規範？

你不用懂那一行的語法。問完之後，Claude 會自己對照規範告訴你它違不違規。

這份手冊怎麼用

每個階段分成兩塊：

```
┌ 📖 先讀懂 ──────────────────────
│ 🎯 這一步的重點
│ 🧭 這一步在做什麼
│ 🧩 它在建構流程的位置
│ ⚖️ 這一步的判斷（有些階段才有）
│ 💡 這一步的指令要包含什麼
└──────────────────────────────

┌ 🛠️ 動手做 ──────────────────────
│ 🧰 這一步的產出
│ 📋 貼給 AI
│ ✅ 怎麼知道做對了
│ 🆘 如果沒有成功
└──────────────────────────────
```

先把「先讀懂」整段看完，再開始動手。

這兩塊的作用不一樣：前面那塊是讓你知道自己在幹嘛，後面那塊才是操作。只照著操作走也做得出網站。但換一個題目，你就不知道要怎麼辦。

每個階段開頭會標示預估時間，例如：

```
📖 讀完約 10 分鐘  🛠️ 動手約 25 分鐘
```

「動手」的時間是抓一般狀況。順的話會更快，卡住的話看 🆘。

Prompt 裡的 `〈…〉` 是要換成你自己內容的地方。

階段 1　先看成品，找出「AI 感」是哪些東西

📖 讀完約 5 分鐘　🛠️ 動手約 25 分鐘

> - **這一步一定要懂**：先自己並排比對兩個版本，寫下差別，寫完才看參考答案。最後把清單補進 `spec.md`。
> - **這一步可以先不懂**：這些元素是怎麼用程式做出來的。這一步不寫任何程式碼。
> - **做完後你要能用自己的話回答**：第一版有、完成版沒有的東西，哪三個最明顯？各自換成咖啡店還成不成立？

📖 先讀懂

🎯 這一步的重點

看得出「AI 感」是由哪些具體元素構成的。這些元素都指得出來，也都刪得掉。

🧭 這一步在做什麼

先把成品打開看一次，知道自己要往哪裡走。

課程附件裡有兩個檔案。它們是**同一個品牌**、**同一套技術**做出來的兩個版本，照片有部分不同。主要的差別是有沒有套用設計規範。

檔案	內容
`index.html`	完成版
`_v1-template-feel.html.bak`	第一版，還沒把多餘的東西刪掉之前

把兩個並排打開，你會最直接看到這個案例要處理的問題。

🧩 它在建構流程的位置

這一步不在建構流程上，它是在流程開始前先讓你看到終點。這一步寫下的清單，會補進你的 `spec.md`，階段 5 再變成 `CLAUDE.md` 的禁用清單。

⚖️ 這一步的判斷

兩個版本並排的時候，你看得出差別在哪裡嗎？

先自己找。參考答案在 ✅ 之後，寫完再對照。先看答案的話，你只是在抄一份清單。換一個題材，你還是認不出哪些東西是模板。

💡 這一步的指令要包含什麼

1–1 不需要下指令，你只要用眼睛看。

1–2 要 Claude 改 `spec.md`，指令裡有兩個限制：每一條都要寫成可以檢查的句子；`spec.md` 其他段落不准動。

🛠️ 動手做

🧰 這一步的產出

- 瀏覽器裡打開的兩個版本
- 一份你自己寫下的「AI 感」清單（至少三條），對照參考答案補齊
- `spec.md` 的「不要：」那一行和視覺類驗收條件，補上這份清單

你自己做（1–1：兩個版本並排比對）

這一步不用貼任何指令，用滑鼠就好：

1. 到課程頁面下載 `case-01-vessel.zip`，解壓縮到桌面
2. 資料夾裡的 `index.html`，**直接用滑鼠雙擊就能打開**
3. `_v1-template-feel.html.bak` 先把副檔名的 `.bak` 拿掉，變成 `_v1-template-feel.html`，再雙擊打開（看不到 `.bak`，是因為系統預設隱藏副檔名。 Windows 11 在檔案總管上方按「檢視」→「顯示」→「副檔名」。 Mac 在 Finder〈設定〉勾「顯示所有檔案名稱延伸檔名」。改名時 Windows 會跳出「如果變更副檔名...」的警告，按「是」）
4. 兩個視窗並排，從上捲到下比對
5. 拿一張紙或開一個記事本，寫下「第一版有、完成版沒有」的東西。至少三條，每一條寫出它在畫面的哪個位置

兩個版本的第一個畫面長這樣：

版本	第一個畫面
第一版（v1）	 兩張是同一個品牌。找出 v1 多了哪些東西
完成版	 看橘色出現在幾個地方，標題旁邊有沒有其他字

第一個畫面只是開始。往下捲，第一版還有更多完成版沒有的東西。

> **為什麼這個檔案雙擊就能開？** 因為它是一個單獨的 HTML 檔案，沒有依賴任何其他東西。
>
> **但你接下來要做的版本不是這樣。** 它要先組裝過才看得到。
>
> **名詞：Astro** 是一套做網站的工具。它讓你把網站拆成一塊一塊來寫：第一個畫面、材料、窯燒各是一個檔案。改其中一塊不會動到別塊。寫完之後，它把這些檔案合成一般的網頁，放上網路就打得開。
> 你需要知道到什麼程度：知道組裝這個動作由 Claude 執行，你不用自己打指令。
>
> 兩者用的技術不同，畫面可以做到一樣。差別是你的版本之後改起來比較容易，而且會多出一套規則和一個自動檢查。

✅ 怎麼知道做對了

- [ ] 我兩個版本都打開了
- [ ] 我先寫下至少三個「第一版有、完成版沒有」的東西，才往下看參考答案
- [ ] 我看得出完成版的圓角比較小、沒有卡片

參考答案：第一版的「AI 感」清單（寫完再對照）

這份清單就是「AI 感」。 每一條都指得出畫面上的哪個東西，也都刪得掉。判斷方法只有一個：把品牌換成一家咖啡店。如果這個元素還能原封不動留著，它就是模板帶來的。

編號	元素	為什麼算模板感
①	導覽列右上的膠囊按鈕「預約參觀」＋圓形箭頭徽章	換成咖啡店的「立即訂位」也一模一樣。完成版改成一條底線加箭頭的文字連結
②	分節編號「(01)」＋裝飾英文「ATELIER —」	網站沒有章節導覽，編號只是裝飾。換成咖啡店，「(01)」一個字都不用改
③	標題裡的裝飾英文「, and」	拿掉之後中文標題意思不變。其他段落的 imperfection、make something that lasts 也是同一類
④	第一個畫面的三欄大數字（2011 / 1,240 / 1280°C）	「成立年份＋作品數＋一個技術數字」是形象網站的固定排法。換成咖啡店就是開店年份、杯數、水溫。完成版只留 1280°C，另外兩個移到「材料」那一段當內文
⑤	跑馬燈	一排字自己橫向捲動，把字換成任何品牌名都成立
⑥	橫向輪播（01 / 05 加左右箭頭）	露出下一張的卡片加左右箭頭，換成咖啡店的菜單照片也一模一樣

編號	元素	為什麼算模板感
⑦	大圓角卡片	32px / 24px 的圓角卡片，任何產業的商品列表都長這樣
⑧	膠囊按鈕「MORE DETAILS」	和 ① 同一種按鈕。按鈕上的英文換成咖啡店也不用改
⑨	四欄大數字＋數字滾動	數字從 0 跑到最終值的動畫，換成咖啡店的「開店天數」也一樣跑
⑩	頂部進度條（畫面最上緣那條橘線）	跟著捲動變長，和內容無關，任何網站都能加。它還用掉了只該留給「火」的橘色
⑪	自訂游標	換成咖啡店也一模一樣。截圖看不出來，在第一版上移動滑鼠自己看
⑫	毛玻璃效果	導覽列捲動後變成半透明模糊底，任何網站都能套
⑬	標籤膠囊	作品卡片上的「灰釉」「花器」小膠囊，換成咖啡店就是「拿鐵」「手沖」
⑭	每一段都有的小標	每一段標題上面都有一行編號加英文的小字，換成咖啡店照樣成立
⑮	全站大寫加寬字距	英文全部大寫、字距拉開，是「高級感模板」的通用手法

> **不算分節編號的**：完成版製程那一段的「01 練土」是步驟序號。製程本來就有先後順序，拿掉編號會少資訊。②的「(01)」拿掉，什麼都沒少。

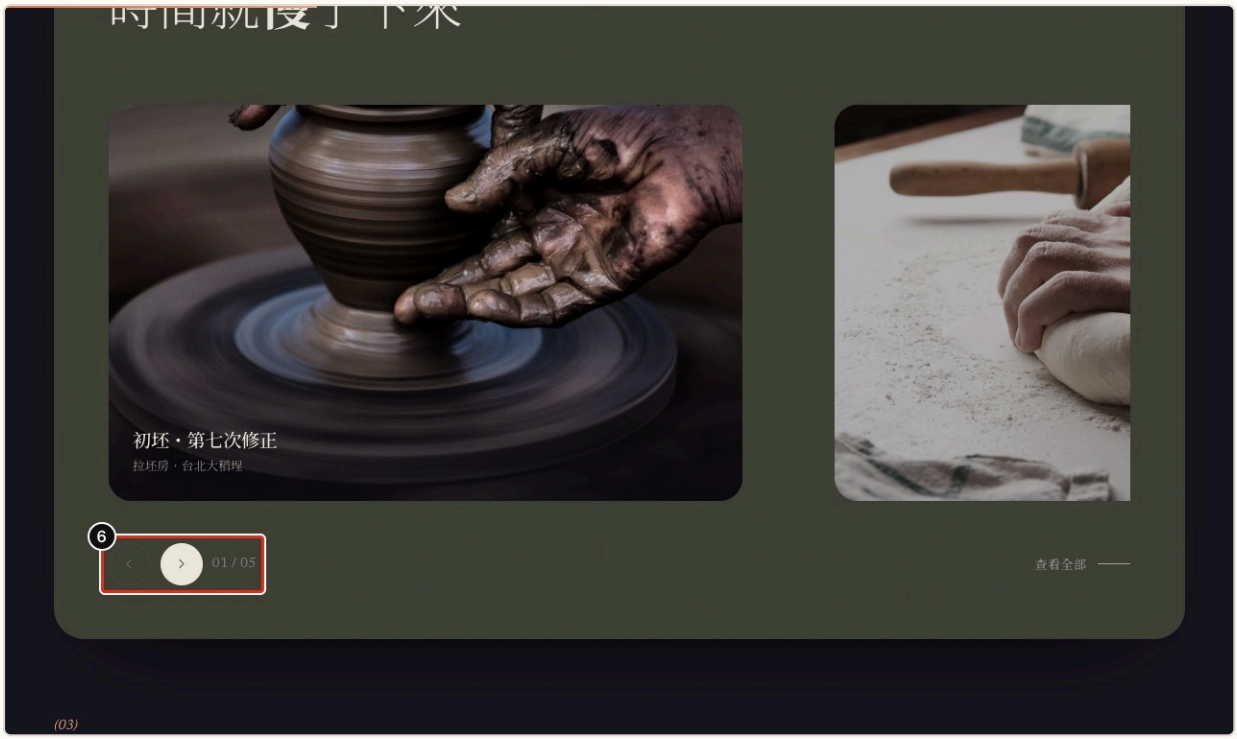
① 到 ④ 在第一個畫面：



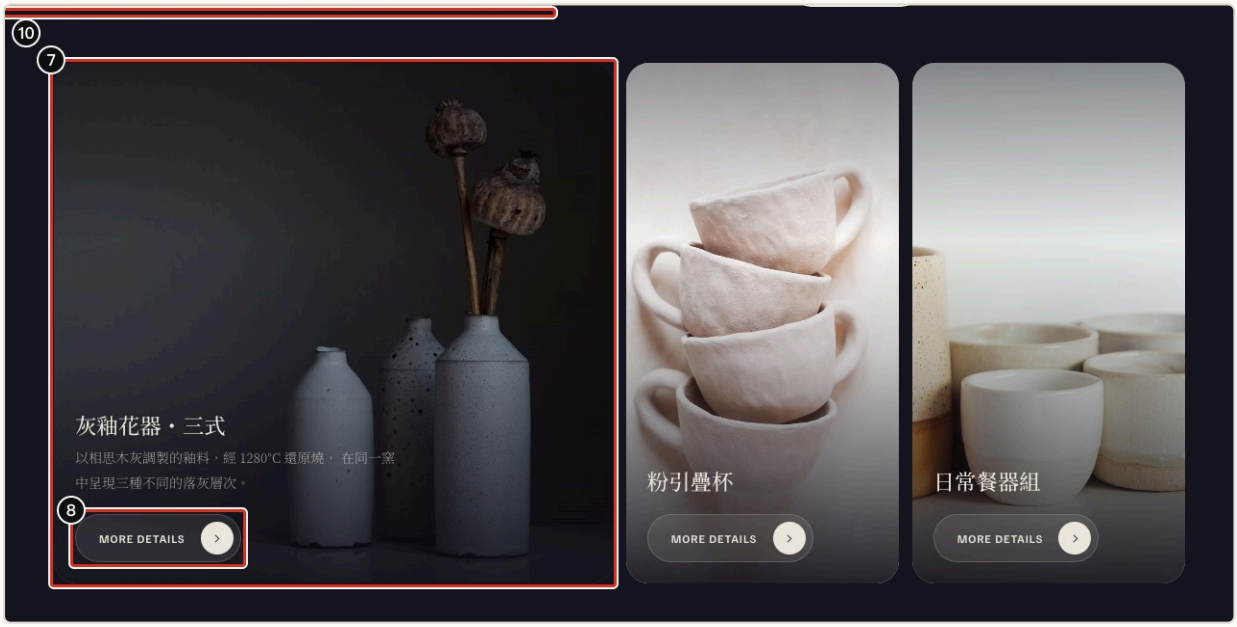
⑤ 跑馬燈：



⑥ 橫向輪播：



⑦ 大圓角卡片、⑧ 膠囊按鈕、⑩ 畫面最上緣的進度條：



⑨ 四欄大數字：



完成版對照：唯一的橘色



完成版的第一個畫面，整個畫面只有一個地方是橘色：右下角紅框裡的 1280°C。印成黑白看不出顏色，看紅框就好。標題是中文自己成立，沒有配一句裝飾用的英文。左下角的「預約工作室參觀」是一條底線加箭頭，不是圓角按鈕。照片沒有被一層黑色蓋住，手上的泥、皮膚的紋理都還看得見，只有文字附近被壓暗。這幾件事後面都會變成明確的規則。

這個案例接下來要做的，就是讓這份清單變成機器檢查得了的規則。

📋 貼給 AI（1–2：把清單補進 spec.md）

對照完之後，把你的清單補進 `spec.md`。課前作業時「不要：」那一行先空著，視覺類的驗收條件也還沒寫，現在補上。

打開 Claude Code Desktop → 切到 **Code** 分頁 → 開一個新對話。 **Project folder** 選課前作業建好的 `vessel`，不是剛剛解壓縮的 `case-01-vessel`。模式選單切到 **Manual**（每一個動作都會先問你）。然後貼上：

貼給 AI

你只需要修改：只有 `〈貼上你的清單〉`：貼你自己寫的、對照參考答案補齊過的那一份。「其他段落一個字都不要動」那一句**照貼**。

```
這是我比對兩個網站版本之後寫下的「AI 感」清單：

〈貼上你的清單〉

請把它整理成可以檢查的句子，補進 spec.md 的兩個地方：
1. 「不做什麼」那一段，「不要：」開頭的那一行
2. 「驗收條件」那一段，加上視覺類的驗收條件

每一條都要能回答「怎麼證明它過了」。
例如「不要跑馬燈」要寫成「頁面上沒有任何自己橫向捲動的文字」。
spec.md 其他段落一個字都不要動。
改完把這兩處改動前、改動後的樣子給我看。
```

Claude 要寫入 `spec.md` 時會跳一張卡片。確認它只改這兩處，再按允許。

做完之後確認：

- [ ] `spec.md` 的「不要：」那一行有內容，驗收條件多了視覺類的條目，其他段落沒有變

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
雙擊之後打開的是程式碼，不是網頁	檔案被文字編輯器接管了	在檔案上按右鍵 → 開啟檔案的應用程式 → 選 Chrome 或 Safari
只寫得出一兩條差別	只看了第一個畫面	兩個版本都從上捲到下，看到卡片、數字、會動的東西就停下來比
Claude 順手改了 `spec.md` 的其他段落	它想幫你把整份潤飾一遍	「請把 spec.md 還原，只改『不要：』那一行和驗收條件，其他段落一個字都不要動」

> **這一步失敗可以跳過嗎**：1–1 可以跳過，它不影響後面任何一步。但最好想辦法看到。後面十一個階段處理的都是這份清單。 1–2 不建議跳過。少了它，階段 5 的禁用清單只能靠 Claude 自己猜。

階段 2　先讓 AI 提計畫，你確認後才動手

📖 讀完約 6 分鐘　🛠️ 動手約 15 分鐘

📖 先讀懂

🎯 這一步的重點

讓 AI 先提出計畫、你確認之後才開始改檔案。這是之後每一個階段的固定開場。

🧭 這一步在做什麼

這一步不做網站，只做一件事：**讓 Claude 先規劃，不要動手。**

Claude Code 跟聊天機器人不一樣。它會真的建立和修改你電腦裡的檔案。所以第一件事是設定邊界：它能在哪裡工作、什麼時候可以動手。

🧩 它在建構流程的位置

流程的起點。你的 `spec.md` 已經在資料夾裡了。這一步是讓 Claude 先理解要做什麼。

💡 這一步的指令要包含什麼

對照操作卡 3，這個 Prompt 用到四個構件：

構件	對應的句子
要做什麼	「我要做一個陶藝工作室的一頁式品牌網站」
不要做什麼	「先不要寫任何程式碼」
限制	「不要超過 400 字」
怎麼算完成	三個編號問題都被回答

三個地方是刻意的：

1. **「先不要寫任何程式碼」**：Plan 模式已經擋住編輯了。但還是明講一次，Claude 的回答才會停在計畫，不會附上程式碼。
2. **「規則 vs 成品」**：規則會影響之後每一次生成，成品是被生成出來的東西。兩個混在一起，你會把該寫進規範的東西寫進單一頁面。後面每一個階段都會回到這個區分。
3. **「不要超過 400 字」**：沒給上限，計畫會長到你讀不完，讀不完就會直接跳過。

🛠️ 動手做

🧰 這一步的產出

- 一個設定好的 Claude Code 對話：Local 環境、Project folder 是 `vessel`、Plan 模式
- Claude 給你的一份專案結構規劃，裡面應該出現 `CLAUDE.md`

📋 貼給 AI

先做四個設定：

1. 確認桌面上有課前作業建好的 `vessel` 資料夾，裡面有 `spec.md`（階段 1–2 剛補過）
2. 確認參考圖在 `vessel/ref/` 裡（階段 3、4 會用到）
3. 打開 Claude Code Desktop → 切到 **Code** 分頁 → 開一個新對話
4. **Environment** 選 **Local**。**Project folder** 選剛剛那個 `vessel`。模式選單切到 **Plan**

注意：選你自己建的 `vessel`，不是剛剛解壓縮的 `case-01-vessel`。

這四個設定等於跟 Claude 說了兩件事。第一，你只能在這個資料夾裡工作。第二，現在只准想，不准動手。

然後貼上：

貼給 AI

你只需要修改：只改第一段的題材。把「陶藝工作室的一頁式品牌網站」換成你 `spec.md` 裡的一句話定義。技術那一行與下面三個編號問題、字數上限，**全部照貼**。

```
我要做一個陶藝工作室的一頁式品牌網站。使用的技術是 Astro + TypeScript +
原生 CSS，最後部署到 Vercel。

先不要寫任何程式碼。請告訴我：
1. 這個專案的目錄結構應該長什麼樣
2. 哪些檔案是「規則」（會影響之後每一次生成），哪些是「成品」
3. 如果我要確保三個月後回來改，設計風格不會偏掉，我應該先建立什麼

用條列回答，不要超過 400 字。
```

> **這幾個技術名詞是什麼**：Astro 在階段 1 有白話解釋，Vercel 在全課地圖下方。TypeScript 是 JavaScript 的一種寫法，可以在網站跑起來之前先抓出一部分錯誤，你不用自己寫。原生 CSS 就是不加任何樣式框架。這裡照抄就好。

✅ 怎麼知道做對了

- [ ] Claude 沒有建立任何檔案（`vessel` 資料夾裡還是只有 `spec.md` 和 `ref` 資料夾）
- [ ] 它的回答裡出現了 `CLAUDE.md` 這個檔案，而且把它歸類成「規則」
- [ ] 我可以用自己的話說出 Plan 模式和 Manual 模式的差別

🆘 如果沒有成功

你可能會看到：Claude 直接開始建立檔案。

通常是因為：模式沒有切到 Plan。也可能是切了之後又開新對話，Plan 模式只套用在當下這個對話。

怎麼辦：把它建的東西刪掉，重新確認模式選單，再貼一次。

這一步失敗可以跳過嗎：不建議。階段 4 也要用 Plan 模式，這一步沒做對，到時候它會直接改檔。

階段 3　建立專案，看懂 AI 要執行什麼

📖 讀完約 7 分鐘　🛠️ 動手約 20 分鐘

📖 先讀懂

🎯 這一步的重點

看懂 Claude 要在你電腦上執行什麼，然後決定要不要允許。

🧭 這一步在做什麼

請 Claude 用官方工具建一個空的網站專案。

指令由它執行，你只要看懂它要做什麼、然後決定允不允許。

> **名詞：專案產生器**（英文叫 scaffold）是官方提供的工具。它會把資料夾結構、設定檔、範例頁面一次建好。
> 你需要知道到什麼程度：知道它預設會塞很多東西進來。下面那段指令用四條負面清單擋掉它們。

這一步還會遇到一個很多人第一次都會困惑的地方：**有一個檔案其實是另一個檔案的捷徑。**

🧩 它在建構流程的位置

還沒進入設計流程。這一步是把資料夾結構建好，讓後面的東西有地方放。

💡 這一步的指令要包含什麼

這個 Prompt 的重點在「不要做什麼」那四行。不要建子目錄、不要範例內容、不要初始化版本控制、不要額外套件。**四條全部是負面清單。**

專案產生器的預設行為會塞很多東西進來。你現在要的是一個乾淨的起點。這正好示範了操作卡 3 為什麼把負面清單排在第二個構件。

最後兩句（列出每個檔案、標註規則或成品）是**怎麼算完成**。你要看得到它做了什麼，不是只看到「完成了」。

🛠️ 動手做

🧰 這一步的產出

- 一個完整的 Astro 專案結構（大約 150MB，大部分是它需要的工具）
- `CLAUDE.md` 與 `AGENTS.md` 兩個檔案，其中一個是另一個的捷徑
- 一筆版本控制的存檔紀錄

📋 貼給 AI

把模式從 **Plan** 切到 **Manual**（每一個動作都會先問你），然後：

貼給 AI

你只需要修改：沒有。整段照貼。那六個條件不用看懂，它們是在擋掉產生器的預設行為。

```
用官方 scaffold 建立 Astro 專案，條件如下：

- 在當前目錄（不要建子目錄）
- 用最乾淨的空白範本（minimal），不要附帶任何範例內容
- 安裝依賴
- 不要初始化 git（我等一下自己來）
- 不要加任何額外的擴充套件（integration）
- 資料夾裡已經有 spec.md 和 ref 資料夾（參考圖），產生器會因為資料夾不是空的而停下來。
  先把資料夾裡現有的東西暫時搬到上一層資料夾，建完再搬回來，內容不要動

指令跑完之後，把 scaffold 產生的每一個檔案列給我，並標註哪些是「規則」哪些是「成品」。
特別注意有沒有產生任何 AI 相關的設定檔。
```

Claude 會先把 `spec.md` 和 `ref` 資料夾搬到上一層，這時會跳卡片。接著它提議執行下面這一行。最後再把它們搬回來：

```
npm create astro@latest . -- --template minimal --install --no-git --skip-houston
```

你不用自己打，但要看得懂它在做什麼。它用官方工具在你選的那個資料夾建立一個網站專案。那個 `.` 就是「這裡」的意思。用最乾淨的範本，順便把需要的東西裝好，先不要接版本控制。

看懂了再按允許。**這個 Prompt 裡真正要你判斷的只有這一行。** 搬檔案的那幾張卡片，確認搬的是 `spec.md` 和 `ref` 資料夾就可以允許。

> ⚠️ 如果 Claude 想加上 `--add none` 這個參數，請它拿掉。那會讓最後一步噴出 `Failed to add integrations` 錯誤。專案還是建得起來，但你會被嚇到。

專案建好、看過它列出的檔案清單之後，接上版本控制：

> 貼給 AI
>
> **你只需要修改**：沒有。整段照貼，引號裡的英文訊息也照抄。
>
> ```
> 初始化 git，建立第一個 commit，訊息就寫 "chore: scaffold astro project"。
>
> 然後跟我確認 .gitignore 有沒有涵蓋 dist/ 和 node_modules/。
> ```

> **名詞：版本控制**（git）是一套記錄「什麼時候改了什麼」的系統。改壞了可以退回去，commit 就是一次存檔。
> 你需要知道到什麼程度：知道它存在就好，指令都由 Claude 執行。

`.gitignore` 是一份清單，列在上面的東西不會存進版本紀錄。`node_modules/` 是裝好的套件，`dist/` 是組裝好的網站成品。兩個都能用指令重新產生，存進去只會讓每次存檔多帶上萬個檔案。

CLAUDE.md 其實是 AGENTS.md 的捷徑

Astro 的專案產生器會自動產生 `AGENTS.md`，並把 `CLAUDE.md` 做成指向它的捷徑。

如果請 Claude 列出這兩個檔案，你會看到類似這樣的東西：

```
AGENTS.md                    874 bytes
CLAUDE.md  ->  AGENTS.md     （這個箭頭代表它是捷徑）
```

裡面是 Astro 自己的技術約定（怎麼啟動預覽、去哪查文件）。

這代表兩件事：

1. 你之後改 `CLAUDE.md`，其實是在改 `AGENTS.md`。這不是壞事。`AGENTS.md` 是跨工具的通用格式，Cursor、Copilot 也讀得懂。但你要知道有這回事。
2. **階段 5 的 `/init` 不會從空白開始。** 它會看到現有內容並往下接。這正好是你要的：Astro 的技術約定該留著，你要加的是設計規範。

> Astro 產生的那份檔案裡有一條很實用的約定：**啟動網站預覽時要用背景模式**，否則 Claude 會被卡在那個一直在跑的程式上，之後什麼都做不了。背景模式是讓預覽在旁邊一直跑，Claude 不用等它結束，可以接著做下一件事。
>
> 在 Desktop 版你不太會遇到這個問題，因為 **Browser** 面板會自己處理。但如果你發現 Claude 突然沒反應了，多半就是這件事。跟它說「請用背景模式啟動」就好。

✅ 怎麼知道做對了

> **名詞：打包**（build）是把你的檔案組裝成瀏覽器看得懂的成品。成品放在 `dist` 資料夾。
> 你需要知道到什麼程度：知道上線的是 `dist` 裡的東西，不是你平常編輯的那些檔案。指令由 Claude 執行，你不用自己打。

把這段貼給 Claude，讓它執行並回報：

貼給 AI

```
請幫我確認三件事，做完把結果告訴我：
1. 把網站打包一次（npm run build），應該成功並產生 dist 資料夾
2. 看一下版本控制的存檔紀錄，應該有一筆
3. 確認 CLAUDE.md 是不是一個指向 AGENTS.md 的捷徑
```

- [ ] 打包成功，出現 `dist` 資料夾
- [ ] 存檔紀錄有一筆
- [ ] 第 3 點的答案是「是」。Windows 上可能是「不是捷徑，是兩個一樣的檔案」，也算對

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
`Failed to add integrations`	指令裡有 `--add none`	請 Claude 拿掉這個參數重跑一次
`command not found: npm`	Node.js 沒裝好	回去看〈環境檢查〉
`EACCES` / `permission denied`	資料夾不給寫	換一個資料夾（桌面上新建一個），重新選 Project folder
裝到一半停住不動	網路擋住了 npm	換網路，或用手機熱點

你可能會看到	通常是因為	怎麼辦
commit 時出現 `Please tell me who you are`	git 還不知道你是誰，新電腦第一次用 git 都會遇到	跟 Claude 說「請幫我設定 git 的使用者名稱與 email，只套用在這個專案」
產生器說 `"." is not empty!`	`spec.md` 或 `ref` 資料夾還在裡面	「請先把 spec.md 和 ref 資料夾暫時搬到上一層，建完再搬回來」
Windows 上 Claude 說 `CLAUDE.md` 「不是捷徑，是兩個一樣的檔案」	這台 Windows 沒有建立捷徑，產生器直接放了兩份內容一樣的檔案	這是正常的。以後都改 `CLAUDE.md` 就好

還是不行的話，把整段錯誤訊息貼給 Claude，再貼這一段：

貼給 AI

請先不要修改任何東西。先告訴我：

1. 最可能是哪三個原因
2. 你準備先檢查哪一個
3. 你需要我提供什麼資訊

一次只處理一個原因。

這一步失敗可以跳過嗎：不行。後面所有東西都建在它上面。

🧭 Checkpoint 1（做完階段 3）

口頭回答就好，三題都講得出來就往下走。

1. Plan 模式和 Manual 模式，各在什麼時候用？
2. 剛剛那個建立專案的指令，你按「允許」之前看懂了什麼？（講出一件就好）
3. `CLAUDE.md` 和 `AGENTS.md` 是什麼關係？改哪一個都一樣嗎？

> 有一題卡住：第 1 題回〈開始之前〉的權限模式那一段，第 2、3 題回階段 3。 **不要靠猜的往下走。**
> 後面九個階段都建在這三件事上。

階段 4　把參考圖變成一份視覺規則

📖 讀完約 10 分鐘　🛠️ 動手約 30 分鐘

📖 先讀懂

🎯 這一步的重點

把一張你喜歡的圖，變成一份可以執行的規則清單。

🧭 這一步在做什麼

把「我喜歡這張圖」變成「我要的視覺規則有這六條」。

「有質感」這種說法 AI 執行不了，「圓角上限 8px」可以。 8px 是檢查上限，這個專案實際用 2px。

這一步分兩段。第一段從參考圖萃取視覺規則（跟題材無關）。第二段把你的 `spec.md` 餵進去決定內容（跟題材有關）。

🧩 它在建構流程的位置

```
你的規格 ──▶ 【視覺規則】 ──▶ 設計規範 + tokens ──▶ 畫面
```

規格之後的第一步。這一步的產出會直接變成階段 5 的輸入。這裡談不清楚，後面的規範就只能寫成形容詞。

⚖️ 這一步的判斷

哪些是本質，哪些只是流行

階段 1 並排看兩個版本時，你已經分過一次。這次對象換成你自己挑的參考圖。

Prompt 4–1 的第 5、6 點就是在逼 Claude 幫你把兩者分開：

- **第 5 點（招牌動作）**：拿掉之後它就不像這個風格的那一兩個特徵。這是本質，要留。
- **第 6 點（設計公式）**：換個題材也照樣成立的東西。這是流行，要刪。

大部分人第一次做 vibe coding，會把整張參考圖照抄。連純粹是模板慣例的東西也一起抄走：分節編號、大圓角卡片、膠囊按鈕、裝飾性英文。抄完之後，成品就會有那種說不出哪裡怪的「AI 感」。那就是你在階段 1 看到的那份清單。

這個區塊該不該存在

Prompt 4–2 的最後一句就是檢驗方式：

> 如果把這些內容換成咖啡、香水、家具還完全成立，那就代表這個區塊寫得太空泛，重寫。

這一條你可以真的拿去試：把文案換成咖啡店，看它成不成立。

💡 這一步的指令要包含什麼

兩個 Prompt 各自示範了一個技巧：

4–1 用「限制輸出格式」逼出具體答案。Prompt 要它「各給我十六進位值」「具體到可以直接寫進 CSS」。有這兩個條件，AI 就沒辦法只回形容詞。

4–2 把整份規格貼進去，而不是描述它。這是操作卡 3 的第三個構件（限制）最實際的用法。與其說「這是一個陶藝網站」，不如把你花 90 分鐘寫的事實全部給它。

🛠️ 動手做

🧰 這一步的產出

- 一組寫死的顏色（含十六進位色碼）
- 字體、版面、元件的規則描述
- `vessel/outline.md`：首頁大綱，有哪些區塊、每個區塊講什麼

顏色和規則**先不用寫進檔案**，留在對話裡就好。階段 5 會整理進 `CLAUDE.md`。在那之前**不要關掉這個對話**，換一個新對話，Claude 就不記得剛剛談定的規則了。首頁大綱要到階段 9 才用，所以 4–3 先存成檔案，和 `spec.md` 放在一起。

📋 貼給 AI（4–1：萃取，不要模仿）

把模式切回 **Plan**。用 Finder（Windows 是檔案總管）打開 `vessel/ref/`。把三到五張參考圖**一起拖進對話框**（或輸入 `@` 再選檔案），然後：

貼給 AI

你只需要修改：只有第一行 `（貼上參考圖）`：把你的三到五張參考圖一起拖進對話框。下面六類的敘述**照貼，不要改成你的題材**，這一步要的就是「跟題材無關的規則」。

（貼上參考圖）

這幾張圖是我喜歡的設計方向。我不要你模仿它們做同樣的網站。我要你把它們拆解成「可以套用到任何題材」的視覺規則。

請萃取出以下六類，每一類都要具體到可以直接寫進 CSS：

1. 色彩：主色、面板色、文字色、強調色，各給我十六進位值
2. 字體：標題和內文各是什麼調性（不用猜字體名稱，描述特徵）
3. 版面結構：欄位比例、對齊方式、圓角大小、留白節奏
4. 元件樣式：按鈕、標籤、圖片容器各長什麼樣
5. 這個設計的「招牌動作」是什麼，就是拿掉之後它就不像這個風格的那一兩個特徵
6. 這個設計裡有哪些元素，其實只是「近年高級網站的公式」，換個題材也照樣成立

第 6 點請特別誠實。不要為了讓我開心而說它都很好。

📋 貼給 AI（4–2：定題材）

這裡要換成**你自己的 `spec.md`。 把中間那一段換成你寫的規格，其餘照抄。**

貼給 AI

你只需要修改：只有中間 `（把你的 spec.md 整份貼在這裡）` 那一行。前後文與最後那條判斷標準（換成咖啡、香水、家具還成不成立）**照貼**。

視覺規則我確認了。現在定題材。

以下是我的規格文件：

（把你的 spec.md 整份貼在這裡，或輸入 @spec.md 引用它）

請用剛剛整理出來的視覺規則，但內容要完全屬於這個題材。
給我一份首頁大綱：每個區塊要傳達什麼、需要哪些素材、
為什麼這個區塊存在。

判斷標準只有一個：如果把這些內容換成咖啡、香水、家具還完全成立，
那就代表這個區塊寫得太空泛，重寫。

注意規格裡「沒有的素材」那幾行。 這幾行最容易被漏掉，代價要到階段 9 才會出現。

以「器 VESSEL」為例。規格裡寫了「沒有窯場的照片」，手上唯一那張其實是金工工坊。少了這一行，Claude 會很自然地規劃一個「窯燒實況」區塊。到時候你只剩兩個選擇：找一張假裝是柴窯的照片，或者整段砍掉重來。

📋 貼給 AI（4–3：存大綱）

大綱談定之後，把模式切到 **Manual**。Plan 模式下 Claude 不會建立檔案。然後貼：

貼給 AI

把剛剛定案的首頁大綱存成 `outline.md`，放在專案根目錄，和 `spec.md` 同一層。
照區塊順序寫：區塊名稱、要傳達什麼、需要哪些素材、為什麼存在。只存大綱，其他檔案不要動。

✅ 怎麼知道做對了

- [ ] 我手上有一組寫死的顏色，每個都有十六進位色碼。只看色碼看不出顏色的話，貼這段請它做預覽：

貼給 AI

把這幾個顏色做成色塊預覽給我看，放在參考圖旁邊。

- [ ] Claude 明確指出了至少兩個「只是流行公式」的元素
- [ ] 每個區塊都有這個題材專屬的數字、工法或地名。沒有「關於我們 / 服務項目 / 聯絡我們」這種區塊

- [ ] 我可以指著任何一個區塊說出「為什麼換成別的產業就不成立」
- [ ] `vessel` 資料夾裡有 `outline.md`，和 `spec.md` 在同一層，打開看得到每個區塊的名稱與順序

🆘 如果沒有成功

你可能會看到：首頁大綱長得像「關於我們 / 我們的服務 / 客戶見證 / 聯絡我們」。

通常是因為：你的 `spec.md` 裡真實素材太少，AI 只能用通用結構填。

怎麼辦：回頭補規格。把這段貼給 Claude：

貼給 AI

```
你剛剛給的首頁大綱，我覺得換成任何產業都成立。

請告訴我：為了讓這份架構只屬於我這個題材，
你還需要知道哪五件事？

一次問我一個問題。
```

它問的問題，就是你規格裡缺的東西。

這一步失敗可以跳過嗎：不行。階段 5 的設計規範和階段 9 的區塊，都直接拿這一步的大綱來做。

階段 5　把視覺規則寫成 AI 每次都會讀的設計規範

📖 讀完約 10 分鐘　🛠️ 動手約 25 分鐘

📖 先讀懂

🎯 這一步的重點

把視覺規則寫成 AI 每次都會讀到、而且**檢查得了**的句子。

🧭 這一步在做什麼

把上一步談定的規則，寫進一個叫 `CLAUDE.md` 的檔案。

你每開一次新對話，Claude 都會先讀它一遍。 所以規則寫在這裡才會持續生效，不是講一次就忘。

你要追加的那一段叫**設計規範**。它管的是這個專案長期都要遵守的規則。

🧩 它在建構流程的位置

```
視覺規則 ──▶ 【CLAUDE.md 設計規範】 ──┐
                                      ├──▶ 第一個畫面
視覺規則 ──▶ Design Token ────────────┘
```

規範管文字規則：不准出現什麼、什麼情況才能用強調色。tokens 管數字，階段 6 會做。兩個檔案寫的是同一批規則：`CLAUDE.md` 寫文字，`tokens.css` 寫數字。

⚖️ 這一步的判斷

這條規則寫得夠不夠具體

判斷方法只有一個：**這條規則能不能被機器檢查，或者被人眼一秒判斷？**

✗ 寫成願望	✓ 寫成規格
圓角不要太大	`border-radius` 上限 8px
顏色要有質感	只用四個色票，強調色佔畫面 < 5%
版面要乾淨	沒有任何兩欄用同一組比例
動畫要順	只有一種進場動畫：1.4 秒淡入 + 14px 位移

常見的 `CLAUDE.md` 寫法是這樣：

> 「本專案採用簡約現代的設計風格，注重留白與質感，營造高級的品牌形象。」

這段話幾乎不會改變 Claude 的產出。「簡約」「質感」沒有一個數字，Claude 只能照自己的預設值做。

這一步寫得不夠具體，到階段 8 就寫不出檢查規則。 那時候你要把這些規則翻成自動檢查，形容詞翻不出來。

💡 這一步的指令要包含什麼

Prompt 5-2 裡有一段「寫作要求」，那五條就是這個階段的全部內容：

1. 不要用形容詞
2. 每一條規則都要能被檢查或一秒判斷
3. 要有明確的禁用清單
4. 強調色要寫**使用條件**，不是只寫色碼
5. 用繁體中文寫

第 4 條特別容易漏。只寫「強調色是 #c4653a」沒有用，AI 會到處拿它當裝飾。要寫「只用於火、窯、溫度、游標焦點，全站佔比 < 5%」。

🛠️ 動手做

🧰 這一步的產出

- `CLAUDE.md`（實際內容寫在 `AGENTS.md` 裡）多出一整段「設計規範」
- 一份明確的禁用清單

📋 貼給 AI（5-1：先跑 /init）

在階段 4 同一個對話裡做。模式維持 **Manual**（4-3 已經切過）。還停在 Plan 的話，先切到 Manual，Plan 模式下 Claude 不會建立或修改任何檔案。

`/init` 是內建指令。它讓 Claude 掃過整個專案，把「這個專案是什麼、怎麼跑」寫進 `CLAUDE.md`。直接在對話框輸入：

```
/init
```

跑完之後 `CLAUDE.md` 會多出專案的說明。實際寫入的是 `AGENTS.md`，因為 `CLAUDE.md` 是它的捷徑。

📋 貼給 AI（5-2：加上設計規範）

還沒決定標題和內文要用什麼字型的話，先在同一個對話貼這段，選好再往下：

貼給 AI

請根據我的參考圖，推薦標題和內文各一個 Google Fonts 字型，並給我看樣字截圖。

貼給 AI

你只需要修改：沒有。整段照貼。五條寫作要求是這一步的全部重點，一條都不要精簡。

現在把設計規範加進 CLAUDE.md。注意 CLAUDE.md 是指向 AGENTS.md 的捷徑，
直接編輯 CLAUDE.md 即可，不要破壞這個捷徑。

保留 Astro 原本的技術約定，在後面追加一個「設計規範」章節。

寫作要求：

1. 不要用形容詞。不准出現「乾淨」「現代感」「高級」「有質感」這種詞。
   這些字對機器沒有意義。
2. 每一條規則都要可以被搜尋，或者被人眼一秒判斷。
   好的例子：「border-radius 上限 8px」
   壞的例子：「圓角不要太大」
3. 要有明確的禁用清單。列出這個專案永遠不會出現的東西。
4. 強調色要寫使用條件，不是只寫色碼。什麼情況可以用、什麼情況不行。
5. 用繁體中文寫。

內容依據我們階段 4 談定的視覺規則。

這是「器 VESSEL」實際使用的版本

這段只看不貼。Claude 會依 5-2 產生你的版本，寫完拿這段對照它有沒有寫數字。

```
## 設計規範

品牌優先於美感。每個視覺元素都要能回答：
這是在表達 VESSEL，還是只是讓網站看起來有設計感？
只有後者的，一律刪除。

### 字體（兩個角色，界線清楚）

| 角色 | 字體 | 用於 |
|---|---|---|
| 標題 | Chiron Sung HK（宋體，可變軸 200–900） | Hero、段落標題、引句、大型數字 |
| 內文 / 介面 | Chiron Hei HK + Familjen Grotesk（拉丁） | 導覽、內文、圖說、小字資訊 |
| 品牌字 | LXGW WenKai TC，只載入「器」一字 | Logo |

禁用：Noto Sans TC、Noto Serif TC、Inter、Space Grotesk、Fraunces、
Instrument Serif、Bricolage Grotesque。
理由：這些是 AI 生成網站的預設值，用了就有 AI 感。

不要讓所有文字都是精品襯線體。內文與介面一律黑體，層次才立得起來。

### 色彩

| 名稱 | 值 | 用途 |
|---|---|---|
| `--ink` | `#141219` | 底色 |
| `--olive` | `#3a3b31` | 窯燒段的整段色塊（滿版、無圓角） |
| `--bone` | `#e9e4da` | 文字 |
| `--ember` | `#c4653a` | 火 |

`--ember` 的使用條件（只有這些）：火、窯、溫度、游標焦點。
全站佔比 < 5%。不做「網站需要一個強調色」的裝飾性使用。

色票只能定義在 src/styles/tokens.css。其他檔案一律引用變數，
不准出現任何十六進位色碼。
```

版面

- 圓角上限 8px（本專案實際值 2px）
- 沒有任何兩欄用同一組比例。現行：材料 .58/.42、窯燒 .52/.40、工作室 .52/.48
- 圖片用 12 欄網格，尺寸與垂直位置都要不同
- 不使用卡片容器、等分網格、置中對齊

動畫

一次慢速淡入（1.4 秒 / 位移 14px）＋ Hero 9% 視差＋ 滑鼠移過時的底線與箭頭位移。
必須支援系統的「減少動態效果」設定。

禁用：逐字動畫、每段都有的捲動動畫、浮動元素、
色塊光暈、3D 旋轉、跑馬燈、自訂游標、進度條、數字滾動。

英文使用規則

英文只出現在有實際資訊價值的位置：VESSEL、ATELIER、EST. 2011。

禁止為了視覺效果加入裝飾性英文：and、crafted、stories、
chapter、collection、imperfection、make something that lasts。

全域禁用清單

分節編號 (01)—(05) / 膠囊按鈕 / 圓形箭頭徽章 / 毛玻璃效果 /
三欄或四欄大數字 / 數字滾動動畫 / 大圓角卡片 / 標籤膠囊 / 橫向輪播 /
每段都有的小標 / 全站大寫加寬字距 / 格狀拼貼版面 / 漸層光暈 / 裝飾性模糊

留白原則

不要因為畫面看起來空就加東西。寧可留白，也不要放不對的素材。

選圖標準

1. 照片必須讓人在一秒內認出「這是陶藝工作室」
2. 每張圖片都要有寬度、高度、替代文字

```
### 最終自我檢查（每次改完設計都要問）

- A：拿掉 logo，還看得出這是陶藝工作室嗎？不像＝品牌素材用得不夠。
- B：把照片換成咖啡 / 香水 / 家具，這套版型還完全成立嗎？
  成立＝設計太模板，重做。
- C：每個「有設計感」的元素拿掉之後，
  品牌辨識下降 → 留；只有華麗程度下降 → 刪。
```

✅ 怎麼知道做對了

- [ ] `CLAUDE.md` 裡沒有任何「乾淨 / 現代 / 高級 / 質感」這類形容詞
- [ ] 有一份明確的禁用清單
- [ ] 強調色寫了使用條件，不是只寫色碼
- [ ] 我可以隨便指一條規則，說出「要怎麼證明它有被遵守」

最後請 Claude 確認捷徑沒被寫壞：

貼給 AI

```
確認一下 CLAUDE.md 現在還是不是一個指向 AGENTS.md 的捷徑。
```

🆘 如果沒有成功

你可能會看到：`CLAUDE.md` 變成一個獨立的檔案，`AGENTS.md` 沒有跟著更新。

通常是因為：某些編輯器在儲存時會把捷徑換成一般檔案。

怎麼辦：

貼給 AI

```
CLAUDE.md 本來是指向 AGENTS.md 的捷徑，現在好像變成獨立的檔案了。
請幫我：
1. 確認現在的狀態
2. 把兩份內容合併成正確的那一份
3. 重新建立捷徑
做之前先告訴我你打算怎麼做。
```

> **這一步失敗可以跳過嗎**：不行。這是後面三個階段的依據。

階段 6　把顏色與尺寸集中成一份設定

📖 讀完約 6 分鐘　🛠️ 動手約 15 分鐘

📖 先讀懂

🎯 這一步的重點

把顏色、間距、圓角這些數字集中在一個檔案，讓它們變成機器檢查得了的東西。

🧭 這一步在做什麼

上一步的規範是文字。這一步把裡面的數字挑出來，集中放在一個檔案。之後全站都引用它們。

> **名詞**：`tokens.css` 裡這些變數的正式名稱叫 **Design Token**（〈開始之前〉介紹過）。
> 你需要知道到什麼程度：知道全站一律引用這些變數，不再寫死數字。

好處有兩個。第一，改配色時只要改一個地方。第二，**規則變成可以被機器檢查的東西**，這是階段 8 的前置作業。

🧩 它在建構流程的位置

```
設計規範（文字規則）──┐
                      ├──▶ 第一個畫面 ──▶ 自動檢查
Design Token（數字）──┘
```

必須在做畫面之前。反過來的話，色碼會散落在十幾個檔案裡，改配色改不乾淨。階段 8 的檢查也會抓到一堆違規。

💡 這一步的指令要包含什麼

三個教學點藏在 Prompt 的細節裡：

1. **「註解寫用途，不寫顏色」**：`/* 底色 */` 有用，`/* 深紫色 */` 沒用。註解要回答「我什麼時候該用這個」。
2. **只載入實際會用到的字**：為了一個字載入整套中文字型是好幾 MB 的浪費。用 `&text=` 參數可以把它降到幾 KB。
3. **中英混排的字體順序**：拉丁字型放前面，中文字型在後。順序反了，英文會被中文字型的拉丁字接管，看起來會怪但很難說出哪裡怪。

> 階段 7 有畫面之後，會回來查這一點。

🛠️ 動手做

🧰 這一步的產出

- `src/styles/tokens.css`：全站唯一可以出現顏色代碼的檔案
- `src/layouts/Base.astro`：網頁的外框，含字體載入設定

📋 貼給 AI

階段 6 到 10 都用 **Manual**。

階段 5 已經選過字型的話跳過這段。還沒選的話，先貼這段，選定後跟 Claude 說「請把 CLAUDE.md 的字體段落改成這兩個」：

貼給 AI

```
請根據我的參考圖，推薦標題和內文各一個 Google Fonts 字型，並給我看樣字截圖。
```

貼給 AI

你只需要修改：只有最後兩行的字體名稱（`LXGW WenKai TC`、要載入的那一個字）換成你自己用的。沒有品牌字就刪掉最後兩行。上面四條規則照貼。

```
建立 src/styles/tokens.css，把設計規範裡的所有色票、圓角、間距、字體堆疊
定義成 CSS 變數。

規則：
- 這是全站唯一可以出現十六進位色碼的檔案
- 每個變數都要有一行註解說明「什麼情況下用它」，不是說明它是什麼顏色
- 間距用 clamp() 定義，不要寫死 px
- 字體堆疊要處理中英文備援（拉丁字型放前面，中文字型在後）

寫完之後在 src/layouts/Base.astro 引入它，並設定好網頁的 <head>：
```

語言、頁面描述、字體預先連線、Google Fonts。

字體只載入實際會用到的字重。LXGW WenKai TC 用 &text= 參數，
只載入「器」一個字。

名詞：`clamp()` 是一個 CSS 函式，寫法是 `clamp(最小值, 想要的值, 最大值)`。
你需要知道到什麼程度：知道畫面變窄時，間距跟著縮小。但間距不會小於最小值，也不會大於最大值。階段 10 的標題字級也用它。

✅ 怎麼知道做對了

把這句貼給 Claude：

貼給 AI

搜尋整個 src 資料夾，看看除了 tokens 檔案以外，
還有沒有任何地方直接寫死了顏色代碼。有的話列出來。

- [ ] 答案是「沒有」
- [ ] `tokens.css` 裡每個變數上面都有一行說明用途的註解
- [ ] `npm run build` 仍然成功（跟 Claude 說「幫我打包一次」，指令由它執行）

只要有一個漏網的，之後改配色就會改不乾淨。

🆘 如果沒有成功

你可能會看到：搜尋結果列出好幾個地方還有寫死的色碼。

通常是因為：Claude 做畫面時直接把色碼寫進了元件。

怎麼辦：

貼給 AI

把剛剛找到的那些寫死的色碼，全部改成引用 tokens.css 裡的變數。
如果某個顏色在 tokens 裡沒有對應的變數，先告訴我，不要自己新增。

最後那句不能省。**多一個顏色，就是多一個設計決定。這個決定要你來做，不是 Claude 自己加。**

這一步失敗可以跳過嗎：不建議。階段 8 的檢查會一直抓到違規。

🧭 Checkpoint 2（做完階段 6）

到這裡你還沒有做出任何畫面，但**規則已經全部定完了**。這是這個案例刻意的順序。

1. 從你的 `CLAUDE.md` 裡挑一條規則，說出「要怎麼證明它有被遵守」。
2. 為什麼顏色只能定義在 `tokens.css` 一個地方？多寫幾個地方會怎樣？
3. 「要有質感」和「圓角上限 8px」，對 AI 來說差在哪裡？

> 第 1 題答不出來，代表那條規則寫成形容詞了。**現在回階段 5 改，比階段 8 才發現便宜得多。**

階段 7　做出第一個畫面，並驗收它有沒有照規範

📖 讀完約 8 分鐘　🛠️ 動手約 30 分鐘

📖 先讀懂

🎯 這一步的重點

一次只做一個區塊。描述視覺時用同一個結構：先說不要什麼、再說要什麼、最後給數字。

🧭 這一步在做什麼

第一次真的產出畫面。

這一步要看的是**它有沒有照著規範走**。好不好看，階段 10 再說。

🧩 它在建構流程的位置

規範和 tokens 的第一次驗證。這一步做完，你就知道前面兩步寫得夠不夠具體。

💡 這一步的指令要包含什麼

這個 Prompt 有兩個地方可以直接搬到你自己的專案。

第一，「先只做 Hero」。

你的直覺會是「一次做完整頁」。一次生成整頁的結果是：你沒辦法判斷哪個決策是好的，也很難改。

> 一次一個區塊 → 看 → 調 → 定案 → 下一個。這是跟 AI 一起工作的基本做法：每一步都小到你還判斷得出來對不對。

第二，「照片處理」那一段的三段式寫法。

```
先說不要什麼　→　不要用一層黑色蓋住整張圖
再說要什麼　　→　用局部漸層：底部、左側、頂部三道
最後給數字　　→　目標比例：照片約 55%，文字約 45%
```

「照片要好看一點」Claude 執行不了，這三段可以。**你之後描述任何視覺效果，都套這個結構。**

🛠️ 動手做

🧰 這一步的產出

- 首頁的第一個畫面（Hero）
- 可以在 Browser 面板裡看到的實際網頁

📋 貼給 AI

先挑 Hero 的照片。照片從 Unsplash 來，和成品一樣直接用圖庫連結，不用下載進專案。到 unsplash.com 用英文搜你的題材。器 VESSEL 用的是拉坯雙手那張 `photo-1590605105526-5c08f63f89aa`。打開照片，在大圖上按右鍵，選「複製圖片位址」。網址要以 `https://images.unsplash.com/photo-` 開頭。 `premium_photo-` 開頭的是 Unsplash+ 付費圖，連結載不出來，換一張。先貼這段，再貼下面做 Hero 的那段：

貼給 AI

```
Hero 的照片用這個 Unsplash 圖庫連結，不要下載進專案，也不要自己換別張：
《貼上 https://images.unsplash.com/photo-… 開頭的網址》
先記下來，等我下一段指令再動手。
```

貼給 AI

你只需要修改：「版面」那幾行裡屬於你題材的文字（標題、地點），以及兩個 `《…》`。**「照片處理」那三行除了 `《…》` 都照貼**：先說不要什麼、再說要什麼、最後給數字。這個結構就是這一步要學的東西。

```
/frontend-design

做首頁的第一個畫面（Hero）。嚴格遵守 CLAUDE.md 的設計規範。

版面：
- 全幅照片打底，滿版高度
- 左上：ATELIER / 陶藝工作室・台北大稻埕（小字資訊）
- 左下：主標題「開窯前，/ 沒人知道結果。」，中文自己成立，不要加英文
- 左下角：連結「預約工作室參觀 →」，下面一行地址與「採預約制」
- 右下：《這個畫面唯一要強調的數字或資訊》，這是這個畫面唯一用強調色的地方

照片處理。這段最重要：
```

- 不要用一層黑色蓋住整張圖。**〈照片裡哪些細節是你品牌的內容〉**才是這個品牌真正的內容
- 用局部漸層：底部、左側、頂部三道，只在文字附近壓暗
- 目標比例：照片約 `55%`，文字約 `45%`

行動呼籲不要用膠囊按鈕或毛玻璃效果。用底線 + 箭頭位移建立互動感。

先只做這一個畫面，其他區塊等一下再說。

「器 VESSEL」的寫法：第一個 `〈…〉` 填「1280°C」，第二個填「手、皮膚、泥土、皺紋」。

> **沒有裝 `/frontend-design` 也可以做這一步。** 它是進階加強，不是必要的。把第一行刪掉，在最前面加上這句再送出：「你現在是一位資深的前端設計師。請嚴格遵守 CLAUDE.md 的設計規範，避免任何一眼看得出是 AI 生成的視覺慣例。」效果會差一點，但流程走得完。

Hero 做好之後，看成果。從 **Views** 選單打開 **Browser** 面板。Claude 會自動偵測並啟動你的網站。拖到對話框旁邊並排，改一次就看得到一次。

如果你想要截圖存檔：

貼給 AI

你只需要修改：只有兩個寬度數字（如果你想看別的尺寸）。其餘照貼。

幫我把網站跑起來，然後截圖給我看第一個畫面在
`1600px` 和 `390px` 兩個寬度下的樣子。

做完之後，390px 寬度（一般手機）應該長這樣：



> **你要看的是什麼**：標題從兩行大字縮成手機讀得下的大小，但**字沒有被壓扁**，中文的筆畫粗細和桌機版一樣。右下角的 1280°C 還在，位置改了但沒有消失。畫面左右不會滑動。

桌機版請對照階段 1 那張圖。

先問自己：第一個畫面左上角那行小字 ATELIER，實際是哪一個字型畫的？這是階段 6 那條字體順序的驗收。用 Chrome 的開發者工具選取那幾個字。 Computed 分頁最下面的 Rendered Fonts 會列出實際用到的字型。也可以請 Claude 查：

貼給 AI

```
請檢查 tokens.css 的字體堆疊，拉丁字型有沒有排在中文字型前面。
再告訴我第一個畫面左上角的 ATELIER 實際會用哪一個字型顯示。
```

查完再對照：器 VESSEL 的內文堆疊是 Familjen Grotesk 在前、Chiron Hei HK 在後。所以那裡應該出現 Familjen Grotesk。

✅ 怎麼知道做對了

- [ ] 照片的細節看得到（不是被一層黑色蓋住）
- [ ] 文字在照片上讀得清楚
- [ ] 沒有膠囊按鈕、沒有分節編號、沒有三欄大數字
- [ ] 390px 寬度下，畫面不會左右移動（沒有橫向捲動）
- [ ] 我可以指著畫面說出至少兩處「這裡有照設計規範走」

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
Browser 面板一片空白	網站還沒啟動完	等 10 秒重新整理；還是不行就跟 Claude 說「請用背景模式啟動網站」
畫面出現膠囊按鈕、大圓角	規範沒有被讀到，或寫得不夠具體	貼下面那段給 Claude
照片整張變暗	用了整層遮罩	貼下面那段給 Claude

貼給 AI

```
這個畫面違反了我的設計規範。請對照 CLAUDE.md 的設計規範章節，
逐條列出你這次的產出違反了哪幾條，然後一條一條修正。
先列清單給我看，我確認之後你再改。
```

這一步失敗可以跳過嗎：不行，但**做得不完美可以先往下**。階段 10 會再回來修。

階段 8　加上自動檢查，讓違反規範的改動被退回

📖 讀完約 10 分鐘　🛠️ 動手約 30 分鐘

📖 先讀懂

🎯 這一步的重點

說得出「寫在 `CLAUDE.md` 的規則」和「會被自動檢查的規則」差在哪裡。然後讓 Claude 裝一個，你親眼看它擋下違規。

🧭 這一步在做什麼

`CLAUDE.md` 是「請你這樣做」。這一步要加的是「做完之後自動檢查一遍」。

> **名詞**：hook 是「自動檢查」的正式名稱，指 Claude 改完檔案之後自動執行的檢查。
> 你需要知道到什麼程度：知道這一步要裝的就是它。違反了就把訊息送回去，要求 Claude 自己修正。

🧩 它在建構流程的位置

```
第一個畫面 ──▶ 【自動檢查】 ──▶ 完成其餘區塊
                  ▲
                  └── 規則在這裡從「請求」變成「會被驗證」
```

這個順序不能反過來。 先裝檢查，後面的區塊才會被保護。反過來的話，你要自己一個一個看。

為什麼需要它

`CLAUDE.md` 寫得再好，Claude 還是可能在第 47 次編輯時漏掉某一條。這不是它偷懶。對話變長之後，注意力會被稀釋。

自動檢查不一樣。它由 Claude Code 這個工具執行，不是由 Claude 執行。所以 Claude 沒辦法忽略它。

運作方式：

```
Claude 改完檔案
      ↓
自動檢查啟動
      ↓
  ┌─ 通過 → 繼續
  └─ 沒通過 → 把違規訊息送回給 Claude → Claude 自己修
```

💡 這一步的指令要包含什麼

Prompt 8–1 有一句不能刪：

> 錯誤訊息用繁體中文，而且要說明「為什麼」不只是「違規了」

那個訊息是要送回給 Claude 看的。**它需要知道理由才能修對。** 否則它可能會把違規的那一行直接刪掉了事。

你寫給 Claude 的其他規則也一樣：**附上理由，它才知道要改哪裡、哪裡不能動。**

🛠️ 動手做

🧰 這一步的產出

- `scripts/design-lint.mjs`：檢查腳本
- `.claude/settings.json`：把檢查掛上去的設定
- 一次「檢查擋下 Claude，然後 Claude 自己修好」的完整過程

快速通道

課程附件 `case-01-vessel.zip` 裡有一個 `teaching/reference/` 資料夾。裡面是**已經測試過的成品**：檢查腳本、設定檔、格式化設定。

如果你想省時間，或者 Claude 生成的版本跑不起來，可以直接用它：

貼給 AI

```
課程附件解壓縮在桌面的 case-01-vessel 資料夾，
裡面的 teaching/reference/ 有現成的檢查腳本與設定。
請幫我把 scripts/、.claude/、.prettierrc.json 複製到這個專案，
然後安裝需要的工具（prettier、prettier-plugin-astro、@astrojs/check、typescript）。
裝完告訴我要怎麼確認它有生效。
```

⚠️ `reference/` 裡的 `design-lint.mjs` 是照「器 VESSEL」寫的。錯誤訊息裡寫的是 VESSEL 的字體與色票名稱。不改的話，Claude 會照著別人的字體與色票去修你的檔案。複製完貼這段：

貼給 AI

```
請把 design-lint.mjs 每條錯誤訊息裡寫死的字體、色票名稱、圓角數字，改成我 CLAUDE.md 和
tokens.css 裡的實際值。
```

但建議你先自己跑一次 8-1 和 8-2。 直接複製檔案，你看不到檢查擋下 Claude 的那一刻。

📋 貼給 AI（8-1：寫檢查腳本）

貼給 AI

你只需要修改：只有規則 1、2 的字體名稱，換成你自己規範裡的禁用字體。**其餘（行為規格五條、規則 3–8、最後兩句）照貼，不用看懂。**

```
建立 scripts/design-lint.mjs，用來在 Claude 每次改完檔案後自動檢查。

行為規格：
- 從標準輸入讀 Claude Code 傳來的資料（JSON 格式）
- 取出被修改的檔案路徑
- 只檢查 .astro / .css / .ts / .js / .mjs，其他直接放行
- 檔案讀不到就放行（不要讓檢查本身變成故障點）
- 不要檢查 scripts/ 資料夾（腳本本身就寫著禁用字體的名稱，否則之後改規則會被自己擋下）
```

檢查規則（違反就回報失敗，並把每一條違規印出來）：

1. 出現 Noto Sans TC 或 Noto Serif TC
2. 出現 Inter / Space Grotesk / Fraunces / Instrument Serif / Bricolage Grotesque
3. border-radius 大於 8px
4. 出現 backdrop-filter（毛玻璃效果）
5. letter-spacing 大於 0.15em
6. 出現十六進位色碼（tokens.css 除外）
7. <img> 沒有 width
8. <img> 沒有 alt

錯誤訊息用繁體中文，而且要說明「為什麼」不只是「違規了」。
這個訊息是要給 Claude 看的，它需要知道理由才能修對。

最後一行加上：「請直接修正，不要詢問。完整規範見 CLAUDE.md。」

📋 貼給 AI（8-2：掛上檢查）

貼給 AI

你只需要修改：沒有。**那一長串指令一個字都不要動**，它是在處理資料夾路徑有空格的情況，改了就會失效。

建立 .claude/settings.json，掛兩個在「改完檔案之後」執行的檢查，
對象是 Write 和 Edit 兩種動作：

第一個：跑 prettier 格式化

```
  jq -r '.tool_response.filePath // .tool_input.file_path' | { read -r f; npx prettier --write --ignore-unknown "$f"; } >/dev/null 2>&1 || true
```

第二個：跑 node scripts/design-lint.mjs

這兩個檢查會同時跑。prettier 只改空白與換行，不會影響第二個檢查的結果。

同時安裝 prettier、prettier-plugin-astro、@astrojs/check、typescript
作為開發用工具，並建立 .prettierrc.json 讓它能處理 .astro 檔。

上面那一長串是什麼：那是 Claude 要執行的指令，你不用看懂，整段複製就好。它的作用是「拿到剛剛被改的檔案路徑，然後對它跑一次格式化」。（裡面的 `jq` 是一個讀取資料格式的小工具。macOS 15 以後內建；更舊的 macOS 或 Windows，看下面那段。）

如果你好奇為什麼寫得這麼繞：macOS 的資料夾路徑常常有空格，例如 `~/Desktop/Vibe Coding Cases/`。用比較直覺的寫法，會在遇到空格時拆錯。

更舊的 macOS 或 Windows，先問 Claude：

貼給 AI

我的電腦有沒有 jq？沒有的話 prettier 那個檢查會安靜地失效，請幫我處理。

⚖️ 這一步的判斷

檢查寫得太粗，會連本來正確的寫法一起擋掉。

例子：規範裡有一條「不要全站大寫加寬字距」，那是要禁掉「高級感模板」的手法。但如果檢查腳本寫成無差別的「`letter-spacing` 不准超過 0.15em」，就會誤傷 `SIGN IN`、`EST. 2011` 這種小字級英文標籤。那些標籤本來就需要寬字距才好讀。

後果不是「檢查沒用」。**AI 會去修一個本來正確的東西**，而且照著錯誤的規則一路修下去。

你在 8–1 寫的第 5 條，就是這種無差別的寫法。

所以每寫一條規則，多問一句：

貼給 AI

這條規則會不會誤擋到本來正確的寫法？
請舉三個「符合規則的意圖、但會被這條規則擋下來」的例子。
如果有，請把條件收窄。

正確的收窄方式是**加條件，不是放寬數值**。以上面那條為例，改成「同時有大寫轉換、字距超過 0.15em、而且字級 14px 以上」才擋。

📋 貼給 AI（8-3：證明它真的會擋）

貼給 AI

你只需要修改：沒有。整段照貼。這一步的重點是親眼看到「擋下 → 收到訊息 → 自己修好」。

```
現在測試檢查是不是真的有效。

請故意在任何一個 .astro 檔的 <style> 裡（例如第一個畫面那個元件），
加入一條 border-radius: 24px。

我要看檢查攔截你、你收到錯誤訊息、然後你自己修正的完整過程。
修完之後把測試用的那條規則也移除。
```

對話裡會出現一段以「設計規範違規」開頭的訊息，下一行寫著違反的是哪一條，例如：

```
[radius/no-giant] border-radius 上限 8px。大圓角卡片是 template 的招牌，這個專案的圓角是 var(--r) = 2px。
```

接著 Claude 會自己修掉那一條。看到這兩件事，就是檢查真的有在擋。

✅ 怎麼知道做對了

- [ ] 我親眼看到 Claude 被檢查擋下來，然後自己修好。這一項是這個階段的重點
- [ ] 我問過「這條規則會不會誤擋正確的寫法」，而且看過它舉的例子
- [ ] 檢查腳本存在於 `scripts/design-lint.mjs`
- [ ] `.claude/settings.json` 存在

還可以直接驗一次腳本本身：

貼給 AI

建一個違反規則的測試檔案（不要放在 `scripts/` 資料夾裡，那個資料夾不在檢查範圍），直接把它餵給 `design-lint` 腳本，
讓我看它的輸出和結束代碼。看完之後把測試檔案刪掉。

- [ ] 結束代碼是 **2**（代表「有問題，擋下來」）

是 0 的話代表腳本根本沒抓到，那這個檢查就是裝好看的。

🆘 如果沒有成功

最常見的狀況：檢查完全沒反應。

通常是因為：Claude Code 只會監看「對話開始時就存在的資料夾」。如果你是在對話進行到一半才建立 `.claude/`，它不會生效。

怎麼辦：在對話框輸入一次 `/hooks`（會重新載入設定），或者關掉對話重開一個。**這是這個階段最容易卡住的地方。**

其他狀況	怎麼辦
腳本執行時報錯	整段錯誤訊息貼給 Claude，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」
檔名有空格導致失敗	確認 8-2 那段指令有整段複製，沒有漏掉引號
檢查太嚴格，正常的東西也被擋	跟 Claude 說「第 N 條規則太嚴格，請改成只檢查〈具體情況〉」

這一步失敗可以跳過嗎：**可以，但階段 9 你要自己一條一條看。** 真的裝不起來就先往下做，之後回來把 8-2、8-3 重做一次。

階段 9　在自動檢查的保護下完成其餘區塊

📖 讀完約 8 分鐘　🛠️ 動手約 40 分鐘

📖 先讀懂

🎯 這一步的重點

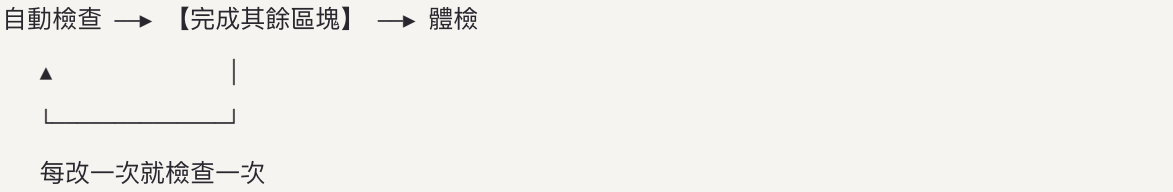
在自動檢查的保護下快速產出，並且做出「拒絕放素材」的決定。

🧭 這一步在做什麼

把剩下的區塊做完。

Claude 寫出違規的東西時，自動檢查會擋下它，Claude 自己改好，不用你出手。

🧩 它在建構流程的位置



⚖️ 這一步的判斷

這張照片該不該放

Prompt 9-2 裡有一句：

> 如果某個區塊你找不到真正對的照片，就不要放照片。

這一句就是「該不該放」寫進 Prompt 的樣子。

參考成品的「窯燒」區塊一度完全沒有照片。手上唯一的窯場素材其實是**金工工坊**（銅壺、鍛台）。與其放一張假裝是柴窯的照片，不如讓那一段只有字。後來找到真正的柴窯照片，才把圖放回去。

💡 這一步的指令要包含什麼

「每個區塊的欄位比例都要不一樣」是**用來擋掉等分網格的規格**。

每一欄都一樣寬，通常只是工具的預設值，沒有人真的挑過。「手工藝品牌需要有控制的不規則」這句話對 AI 沒有意義。「沒有任何兩欄用同一組比例」才有。

這就是把形容詞翻成規格的實際範例。

先問自己：你的每個區塊，左右兩欄各佔多少？畫面上只看得出哪一欄比較寬，比例的數字要查程式。每做完一個區塊，貼這段：

貼給 AI

請列出每個區塊左右兩欄的比例，有兩個區塊用同一組的標出來。先不要改。

🛠️ 動手做

🧰 這一步的產出

- 完整的首頁：`outline.md` 裡的每一個區塊
- 過程中至少一次「自動檢查擋下違規」的紀錄

📋 貼給 AI（9–1：其餘區塊）

「首頁大綱」就是階段 4–3 存的 `outline.md`。第一個 `〈…〉` 照那份檔案的區塊順序填，或直接寫 `@outline.md`。

貼給 AI

你只需要修改：只有兩個 `〈…〉`。中間「共通要求」那四條照貼，它們是在對抗模板慣例。

接著做其餘區塊。順序照首頁大綱：**〈換成你自己的區塊順序〉**。

一次做一個，做完給我看再繼續。

共通要求：

- 每個區塊的欄位比例都要不一樣（規範規定）
- 至少一張圖要切出視窗邊緣
- 不要有卡片容器
- 區塊編號只存在於大綱裡，不要印在畫面上

〈如果有某個區塊特別重要，在這裡單獨說明它的處理方式〉

「器 VESSEL」的版本是這樣寫的，可以參考：

> 「窯燒」這一段特別說明：
> 用滿版的 `--olive` 色塊，無圓角。上方放一張窯內照片作為橫幅。
> 這是全站唯一換底色的地方，因為那是窯本身。強調色只用在 1280°C。
> 內容是溫度時序（08:00 → 950°C → 1280°C → 72 小時），不是行銷文案。

📋 貼給 AI（9-2：圖片）

Hero 那張在階段 7 已經挑好，這裡是其他區塊的圖。規則和階段 7 一樣：連結要 `photo-` 開頭，Claude 給了 `premium_photo-` 的就請它換掉。

貼給 AI

你只需要修改：只有 `（你的題材）` 這一處。其餘照貼。尤其「找不到對的照片就不要放照片」那三行。

```
所有圖片先用 Unsplash 的圖庫連結。

選圖標準（設計規範的「選圖標準」那一節）：
照片必須讓人在一秒內認出「《你的題材》」。

如果某個區塊你找不到真正對的照片，就不要放照片。
寧可留白，也不要用一張看起來差不多但其實不對的圖。
如果你這樣決定了，明確告訴我是哪一段、為什麼。

每張圖片都要有寬度、高度、替代文字。
```

選好圖之後問自己：「窯燒」那一段的照片裡看得到窯嗎？畫面上是磚砌窯道和成排陶坯，還是銅壺、鍛台？區塊叫「窯燒」，照片就要拍到窯。圖是 Claude 挑的話，請它逐張描述：

貼給 AI

```
請逐張描述每一張圖畫面裡實際看得到的物件，不要寫你預期它是什麼。
再對照它放的區塊，告訴我哪幾張對不上。沒有實際看過的圖，直接說沒看過。
```

做完之後，各區塊應該接近這樣

這四張是「器 VESSEL」的成品。**不用做得一模一樣。** 要看的是每一張下面那句話講的原則。



> **你要看的是什麼**：左右不是各佔一半（這裡是 .58 / .42），右邊那張照片切出了視窗右緣、沒有被容器框住。下面那排數字（2011 工作室成立 / 1,240 件出窯作品 / 3 處土源）是內文級的小字，不是三欄大數字。同樣的資訊，放大成三欄就會變成模板公式。



你要看的是什麼：照片是滿版橫幅，沒有圓角、沒有陰影、沒有卡片外框。下面那片橄欖綠是整段色塊，直接切齊視窗兩側。 **整個網站只有這一段換底色**，因為它講的是窯。



你要看的是什麼：右邊那一排是溫度時序，不是「我們的優勢」。四列裡只有 1280°C 是橘色，那是全站的最高溫，也是規格裡寫的「唯一會用強調色的數字」。其他三列都是白色，因為它們只是過程。

先問自己：你的頁面上，橘色出現在幾個地方？每一處都和火、窯、溫度或焦點有關嗎？器 VESSEL 用到橘色的是：

- Hero 的 1280°C
- 溫度時序的最高溫
- 製程最後一步的 1280°C
- 滑鼠移過的底線
- 鍵盤焦點的外框
- 選取文字的底色

自己數容易漏，請 Claude 列：

貼給 AI

請列出所有用到強調色變數的地方，每一處註明它屬於火、窯、溫度、焦點的哪一個。

哪一處都不屬於的標出來，先不要改。



你要看的是什麼：這一段的左右比例又跟前面兩段不一樣（.52 / .48），而且圖片換到左邊。**沒有任何兩個區塊用同一組比例**，這是規範裡那條「沒有任何兩欄用同一組比例」的實際樣子。照片下面那行小字（二樓素坯架・等待進窯的三百多件坯體）是圖說，有了這行圖說，照片就有了具體的位置和數量：二樓、三百多件坯體。

✅ 怎麼知道做對了

- [ ] 所有區塊都完成了
- [ ] 沒有任何兩欄用同一組比例
- [ ] 過程中自動檢查至少擋下一次違規（一次都沒被擋，代表 Claude 都照規範寫，8–3 那一次也算）
- [ ] 每一張照片我都說得出「為什麼是這一張」
- [ ] 拿掉 logo，我還認得出這是什麼行業

🆘 如果沒有成功

狀況一：Claude 一次把所有區塊都做完了

通常是因為：「一次做一個」那句被忽略了。

怎麼辦：不用全部重來。逐區塊檢查：

貼給 AI

```
你一次做完了全部區塊，我沒辦法逐一判斷。
請從第一個區塊開始，一個一個告訴我：
這個區塊的欄位比例是多少、用了哪張照片、為什麼是這張。
一次講一個，我確認之後再講下一個。
```

狀況二：說找不到適合的照片，或者放上來的圖根本不是那個東西

你要的是窯，它給你烤箱。或者它直接說圖庫裡搜不到。

通常是因為：**它搜到的東西和你搜到的不一樣。** 它說找不到，不代表那張圖不存在。

怎麼辦：自己去 unsplash.com 搜一次。找得到就把網址貼回去，指定它用這幾張：

貼給 AI

```
你說找不到的那幾張，我自己搜到了。請改用這幾個網址：

（一行一個網址，後面註明這張要放在哪一個區塊）

請確認每一張都真的載得出來。載不出來的直接告訴我，不要自己換一張別的。
```

最後那一句不要刪。少了它，載不出來的圖它會默默換成別張，而你要到驗收時才發現照片不對。

> **這一招不只用在找圖。** AI 說「找不到」「查不到」「沒有這個」的時候，先自己查一次。查得到就把結果貼回去給它。Claude 搜得到的範圍比你在瀏覽器裡搜的小。它說找不到，不代表沒有。

這一步失敗可以跳過嗎：不行，這是主要產出。

🧭 Checkpoint 3（做完階段 9）

畫面完成了。這三題確認你不是只做出畫面。

1. 自動檢查擋下 Claude 那一次，是誰發現問題的、又是誰修好的？
2. 你的網站拿掉 logo，還認得出是什麼行業嗎？指出畫面上兩個依據。
3. 有沒有哪一段你選擇留白、不放照片？為什麼那不算偷懶？

> 第 1 題如果答案是「我自己看到的」，回階段 8 確認檢查真的掛上去了。這一階段沒被擋過的話，用 8–3 那一次回答。第 3 題如果沒有任何一段留白，不一定是錯。但回頭看一次階段 9 的〈⚖️ 這一步的判斷〉。

階段 10　量四個尺寸，找出眼睛看不出來的問題

📖 讀完約 8 分鐘　🛠️ 動手約 30 分鐘

📖 先讀懂

🎯 這一步的重點

用工具找出自己看不到的問題，並且**先看清單再決定修哪些**。

🧭 這一步在做什麼

做一次全面的品質檢查：響應式、無障礙、效能、排版細節。

重點在**先列出問題清單，再決定要修哪些**。不要讓 AI 一次全改，你會失去判斷的機會。

🧩 它在建構流程的位置

畫面全部做完之後的第一次品質檢查。

💡 這一步的指令要包含什麼

Prompt 10–1 的最後一句是重點：

> 修改前先列出你找到的問題，我確認之後再動手。

`/impeccable` 會順手改掉你刻意的設計決策，例如不等分的欄位。

分工應該是這樣：

```
AI 負責找問題  →  你負責判斷該不該修  →  AI 負責執行
```

Prompt 10–2 則示範另一件事：**橫向溢出和實際算出來的字級，要量過才知道。**

實際做這個專案時抓到一個典型案例。讓圖片切出視窗右緣的效果**一直沒有生效**，好幾個版本都停在容器邊界。原因是三層 CSS 互相影響，需要三個條件同時成立才會動。

這種問題畫面不會壞掉，只是「看起來有點怪」。 眼睛看很難發現。用工具量一次寬度就看得出來。

先問自己：材料區塊那張照片的右緣，有沒有碰到瀏覽器視窗的右緣？沒生效的樣子是照片右邊還留著一條深色底，寬度和左邊文字到視窗左緣的距離差不多。器 VESSEL 用了三個條件：

- 右邊距設成負的

- 寬度加上同一段距離
- 取消圖片「最大寬度 100%」的限制

少一個，圖就停在容器邊界。請 Claude 量：

貼給 AI

請量材料區塊那張照片的右緣位置，和視窗寬度比較。

沒有切齊視窗右緣的話，列出是哪幾條 `css` 把它限制在容器裡。先不要改。

鍵盤也一樣，要按過才知道。

先問自己：手機選單關著的時候，在桌機上一直按 Tab，焦點會不會跑進那個看不見的選單？器 VESSEL 原本會。按到第 9 下，焦點從畫面上消失，再按 5 下才出來。原因是選單關起來時只用 `clip-path` 遮住，裡面 5 個連結還在。用滑鼠的人完全不會發現。請 Claude 量：

貼給 AI

請在桌機寬度從頁首開始按 `Tab`，記下每一下焦點停在哪個元素，以及那個元素在畫面上看不看得到。

看不到的列出來，先不要改。

🛠️ 動手做

🧰 這一步的產出

- 一份問題清單，以及你對每一項的決定（修 / 不修）
- 四個尺寸的實測數據與截圖

📋 貼給 AI（10–1：體檢）

貼給 AI

你只需要修改：只有第 1 點的四個尺寸（如果你要支援的寬度不一樣）。四個檢查面向與最後那句「先列出問題，我確認之後再動手」照貼。

```
/impeccable

對整個首頁做一輪整體檢查與修正。重點順序：

1. 響應式：1600 / 1280 / 834 / 390 四個尺寸都要檢查
   特別注意中文大標題，必須用 clamp()，不准用 transform: scaleX()，
   不准為了塞進畫面壓扁字面
2. 無障礙：對比度、鍵盤焦點樣式、語意標籤、支援「減少動態效果」
3. 效能：圖片尺寸、字體載入策略、畫面載入時的位移
4. 排版細節：中文換行有沒有孤字（一行只剩一兩個字）

修改前先列出你找到的問題，我確認之後再動手。
```

沒有裝 `/impeccable` 也可以做這一步。 它是進階加強，不是必要的。把第一行刪掉，在最前面加上這句：「請對整個首頁做一次完整的品質檢查，涵蓋響應式、無障礙、效能、排版細節四個面向。」檢查會比較淺，但四個重點還是會走到。

📋 貼給 AI（10-2：真的量一次）

貼給 AI

你只需要修改：只有四個寬度數字。其餘照貼，包含最後那句「需要先安裝工具的話先告訴我」。

幫我用瀏覽器自動化工具，在 1600 / 1280 / 834 / 390 四個寬度實測，回報：

- 有沒有橫向溢出
- 主標題實際算出來的字級是多少
- 第一個畫面的內容有沒有超出可視範圍
- 標題有沒有被套用變形效果（不該有）

用表格回覆，並存下每個尺寸的截圖。

需要先安裝工具的話，先告訴我要裝什麼，我同意了你再裝。

> **名詞**：**Playwright** 是一個可以自動開瀏覽器、量尺寸、截圖的工具。Claude 會提議安裝它。你需要知道到什麼程度：知道第一次安裝會下載一個瀏覽器，大約幾百 MB。跑一兩分鐘是正常的。這屬於操作卡 2 的第二層：**執行前要你確認**。

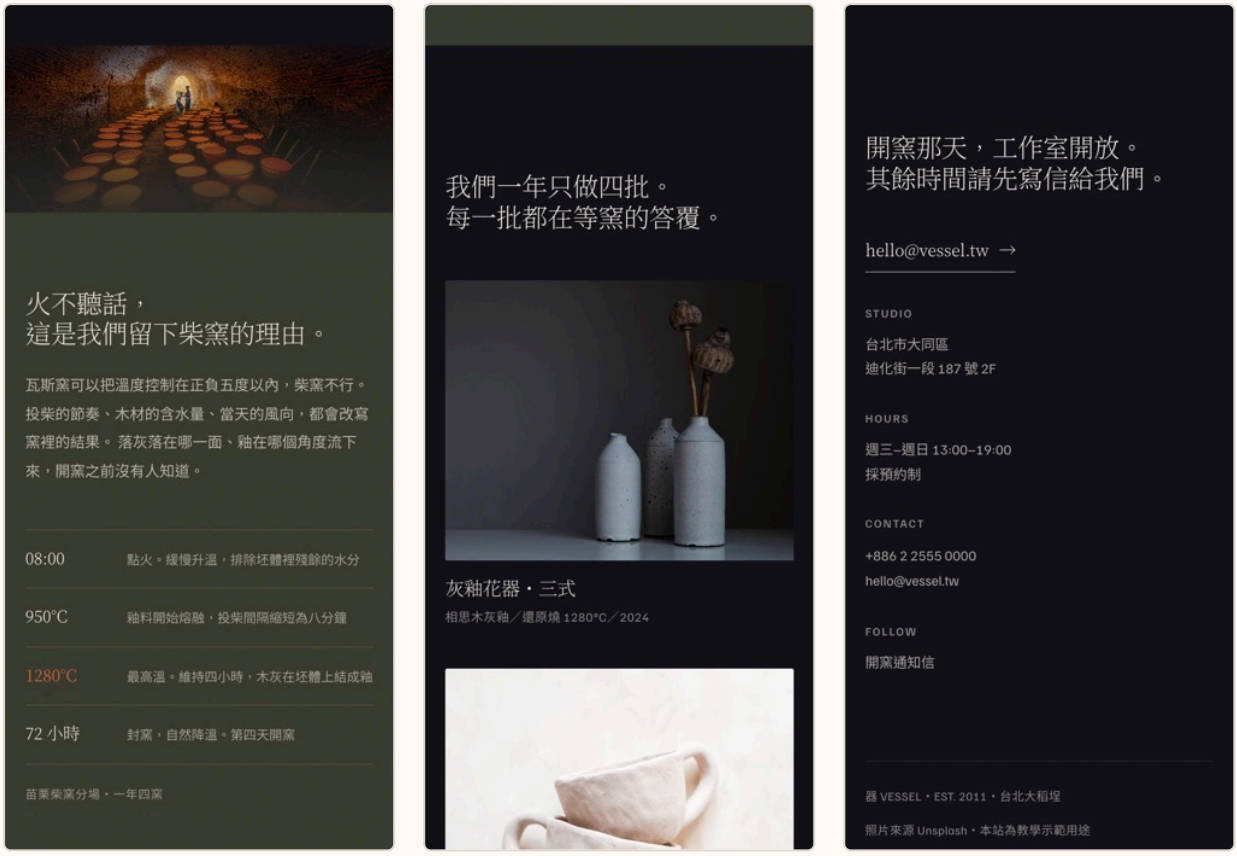
先問自己：主標題在 390px 應該是幾 px？沒先算出來，回報的數字對不對你沒辦法判斷。`clamp()` 的三個數字就是答案。表格回來之後貼這段：

貼給 AI

請把主標題的 clamp() 寫法給我，算出四個寬度各應該是幾 px，再和實測值並排。

算完再對照：器 VESSEL 寫的是 `clamp(46px, 9vw, 128px)`。實測 1600 / 1280 / 834 / 390 分別是 128 / 115 / 75 / 46px。

手機上應該長這樣



> **你要看的是什麼**：三張都沒有橫向捲動，中文標題全部是等比例縮小、沒有被壓扁。左右不對稱的版面在手機上自動變成單欄，但**留白的節奏還在**，區塊之間仍然隔得很開，沒有為了塞進手機就把東西全部擠在一起。

✅ 怎麼知道做對了

- [ ] 我看過完整的問題清單，而且對每一項都做了決定
- [ ] 四個尺寸全部沒有橫向溢出
- [ ] 四個尺寸的標題都沒有被變形壓扁
- [ ] 第一個畫面在四個尺寸下都容納得下
- [ ] 從頁首按 Tab 一路到頁尾，焦點每一下都看得到
- [ ] 我有至少拒絕一項它的修改建議

🆘 如果沒有成功

你可能會看到	怎麼辦
它沒列清單就直接開始改	「請先還原剛剛的修改，改成先列清單給我看」
安裝 Playwright 卡住	網路問題。換網路重試；真的不行就跳過 10–2，用 Browser 面板手動拉寬度目測

你可能會看到	怎麼辦
390px 有橫向捲動	把測量結果貼給 Claude：「請找出是哪一個元素造成的，先告訴我，不要直接改」

這一步失敗可以跳過嗎：10–1 不行，10–2 可以（用目測代替，但精確度差很多）。

階段 11　讓 AI 審查 AI，並學會拒絕它的建議

📖 讀完約 7 分鐘　🛠️ 動手約 20 分鐘

📖 先讀懂

🎯 這一步的重點

判斷 AI 的哪些建議該收、哪些該拒絕，並且說得出理由。

🧭 這一步在做什麼

讓 Claude 審查自己寫的東西，而且是在**一個新的對話**裡審。

寫程式的那個對話裡，留著你們談過的每一個決定。在那裡審查，Claude 會把這些決定當成理所當然，不會提出來。新對話只看得到程式碼和 `CLAUDE.md`。沒寫進 `CLAUDE.md` 的決定，它會當成問題提出來。寫進去的，它應該照規範判斷，但通用的「標準做法」還是常常蓋過規範。兩種情況都要你裁決：這條建議該不該收。

重點是學會**拒絕**。AI 的建議不一定都要聽。這個階段要你說得出「為什麼不收」。

🧩 它在建構流程的位置

上線前的最後一次檢查。

⚖️ 這一步的判斷

這條建議該不該收

`/code-review` 的預設價值觀是通用軟體工程：一致性、可預測性、標準做法。

但這個專案有**刻意的不一致**：不等分的欄位、不對稱的圖片位置。那是設計決策，不是缺陷。

> **當工具的建議和專案的規範衝突時，規範優先。** 而且你要能說出理由。

這也反過來驗證了階段 5。規範寫得夠具體，這種衝突當場就能裁決。規範只寫「要有設計感」，你就只能靠感覺爭論。

💡 這一步的指令要包含什麼

Prompt 11–1 的第二段重點是**先分類，再處理**。

直接說「幫我修掉 review 找到的問題」，AI 會把所有東西都改一遍。連和你的設計決策衝突的那幾條也一起改。分成三類之後，你才有機會逐項裁決。

🛠️ 動手做

🧰 這一步的產出

- 一份分成三類的問題清單
- 至少一條你明確拒絕的建議，以及拒絕的理由
- 一次乾淨的打包與型別檢查

📋 貼給 AI（11–1：在新對話裡跑審查，分類再處理）

先開一個新對話。切到 **Code** 分頁，開一個新對話，**Project folder** 選同一個專案資料夾。 11–2 也在這個新對話裡做。

`/code-review` 是內建的，不用另外安裝。在新對話的對話框輸入：

```
/code-review src
```

後面那個 `src` 是要它審整個 `src` 資料夾。只打 `/code-review` 的話，它審的是「上一次存檔紀錄之後改過的部分」。上一次存檔之後才新建的檔案，可能會被漏掉。

先問自己：你上一次存檔是在哪個階段？之後新增的檔案有沒有在 `src` 裡？不確定的話，在這個新對話先貼這段：

貼給 AI

```
請列出上一次 commit 之後新增或改過的檔案。先不要修改任何東西。
```

清單裡有 `src` 底下的檔案，就照上面輸入 `/code-review src`。

審查跑完之後，貼這段讓它把問題分類：

貼給 AI

你只需要修改：只有最後那個例子（等分網格）換成你自己規範裡的一條規則。A / B / C 三類的分法照貼。

```
把 review 找到的問題分成三類再給我：

A. 真的會壞，現在就修
B. 可以更好，值得修但不急
C. 風格偏好，和我們的設計規範衝突，不修

C 類要特別說明為什麼衝突。

例如：review 如果建議把區塊改成等分網格「比較整齊」，
那就是 C，規範明確規定「沒有任何兩欄用同一組比例」。
```

📋 貼給 AI（11-2：補上型別檢查）

貼給 AI

你只需要修改：沒有。整段照貼。

```
把 astro check 加進 package.json 的 scripts，然後跑一次。
有錯就修掉。
如果還沒裝 @astrojs/check，連同 typescript 一起裝，typescript 用 6 版
（astro check 目前不支援 TypeScript 7）。

同時確認打包是乾淨的：npm run build
```

名詞：**型別檢查**是一種在網站真的跑起來之前，先找出部分錯誤的自動檢查。例如某個元件要的是數字，傳進去的卻是文字，它會指出是哪個檔案、哪一行。
你需要知道到什麼程度：知道它跑過、而且沒有錯誤就好，你不需要會寫。

✅ 怎麼知道做對了

貼給 AI

把網站打包一次，並且做一次型別檢查。兩個都要沒有錯誤。

有錯的話先告訴我是什麼，不要直接改。

- [ ] 打包成功、型別檢查沒有錯誤
- [ ] 審查是在新對話裡跑的，不是在寫程式的那個對話
- [ ] 問題被分成 A / B / C 三類
- [ ] **我至少拒絕了一條 C 類建議，而且說得出理由**

最後那句「有錯的話先告訴我是什麼，不要直接改」不能省。**你要看到問題，才有機會判斷。**

🆘 如果沒有成功

你可能會看到：C 類是空的，所有建議都被歸成 A 或 B。

通常是因為：你的規範還不夠具體，所以沒有東西可以拿來裁決。

怎麼辦：

貼給 AI

C 類是空的。請重新檢查一次：

`review` 的建議裡，有沒有哪一條的理由是「一致性」「標準做法」「比較整齊」？

把它們對照 `CLAUDE.md` 的設計規範，看看有沒有衝突。

這一步失敗可以跳過嗎：11–2（打包與型別檢查）不行，那會影響上線。11–1 可以，但你會少掉一次「這條建議該不該收」的練習。

階段 12　推上 GitHub 並部署成公開網址

📖 讀完約 8 分鐘　🛠️ 動手約 30 分鐘

📖 先讀懂

🎯 這一步的重點

分辨哪些動作可以放手讓 AI 做，哪些要它先說明、你點頭才能做。

🧭 這一步在做什麼

把成品變成一個真的網址，並且**真的量一次**速度，不是憑感覺說「還蠻快的」。

🧩 它在建構流程的位置

流程的終點。

⚖️ 這一步的判斷

回到操作卡 2 的三層原則：

```
讀檔、寫檔、跑測試、本機存檔　→　放手讓它做
安裝新工具、大量刪除檔案　　→　執行前要你確認
建 GitHub 專案、推上去、部署　→　它說明清楚，你點頭才做
```

推上 GitHub 和部署是兩件事。 12–1 建的 GitHub 專案設成 private，推上去只有你看得到。部署才會公開：Vercel 部署完成後，任何人拿到網址都打得開。 12–3 接上 Vercel 之後，每推一次就自動部署一次。從那時起，推上去就等於公開。

重點不是「一定要你親手按那顆按鈕」。Claude 可以幫你建專案、幫你推上去，但它動手之前，你必須先看到它要做什麼。**你要有機會在執行前喊停，這才是重點。**

階段 8 把規則交給自動檢查。這一步反過來：**建專案、推上去、部署，三件事都要你看過指令才執行。**

💡 這一步的指令要包含什麼

Prompt 12–1 的最後一句是這個階段的重點：

> 先把你準備執行的每一個指令列出來給我看，我確認之後你再做。

這是操作卡 3 的第六個構件（哪裡要停下來問我）的實際用法。這句不是禁止 Claude 做，是**要它先把指令列出來**。

🛠️ 動手做

🧰 這一步的產出

- 一個 GitHub 專案
- 一個可以打開的公開網址
- 一份上線後的實測報告

📋 貼給 AI（12-1：先確認，再讓它動手）

貼給 AI

你只需要修改：只有 `〈…〉` 那個專案名稱。**最後那一句「先把你準備執行的每一個指令列出來」不要刪**，那是這個階段的重點。

```
準備部署到 Vercel。

這是純靜態網站（所有頁面都是事先做好的檔案，沒有伺服器在跑），所以：
- 不需要額外的轉接套件
- Vercel 會自動偵測 Astro 並用 dist/ 當輸出目錄

請先確認：
1. npm run build 產出的 dist/ 是完整的
2. .gitignore 有排除 dist/ 和 node_modules/
3. 專案設定裡標示的 Node 版本，Vercel 支援得了

確認完之後，我要你幫我做這三件事：
- 把所有改動做一次 commit
- 建一個名叫 〈vessel〉 的 GitHub 專案，設成 private（Vercel 讀得到私人專案）
- 把程式碼推上去

但先不要動手。先把你準備執行的每一個指令列出來給我看，
每一行說明它會做什麼、會不會產生我收不回來的東西，我確認之後你再做。
```

📋 貼給 AI（12–2：確認之後，讓它執行）

看過它列出來的指令，確認沒有多做你沒要求的事，再回它：

貼給 AI

你只需要修改：沒有。整段照貼。

```
可以，照你剛剛列的做。
建好之後告訴我 GitHub 專案的網址。
```

如果它問你要帳號密碼：不要在對話框裡輸入。先停下來，回到〈開始之前〉的〈接上 GitHub〉做一次。做完再回它：「可以，照你剛剛列的做。建好之後告訴我 GitHub 專案的網址。」

它建立專案的時候，Claude Code 會跳出一個允許視窗。 那就是第三層原則實際運作的樣子：對外的動作，工具會停下來等你點頭。看清楚它要做什麼再按允許。

你自己做（12–3：把 Vercel 接上這個專案）

這一步只做一次。接上之後，**你每次推上 GitHub，Vercel 都會自己重新部署。不用再回來按任何東西。**

1. 到 vercel.com，用 GitHub 帳號登入
2. 選 **Add New Project**
3. 在列表裡找到你剛剛建的專案，把它匯入
4. 設定畫面**什麼都不用改**（Vercel 會自己認出這是 Astro）
5. 等一到兩分鐘，會出現一個 `xxx.vercel.app` 的網址

畫面和上面寫的不一樣：這類服務的介面常常改版。把你看到的畫面截圖貼給 Claude，說「我要把這個 GitHub 專案接到 Vercel 上，現在該點哪裡」，讓它帶你走。

📋 貼給 AI（12-4：上線後驗收）

貼給 AI

你只需要修改：只有 `〈…〉`：換成你的網址。下面四項驗收照貼。

網站已經上線在 **〈貼上你的網址〉**。

用瀏覽器自動化工具對線上版本做最後驗收：

- 四個尺寸（1600 / 1280 / 834 / 390）的截圖
- 所有圖片都有載入成功
- 沒有 console 錯誤
- 量一下第一個畫面的載入時間（LCP）

有問題就列出來，我們再決定要不要修。

✅ 怎麼知道做對了

- [ ] 我可以用手機打開那個網址，畫面正常
- [ ] 四個尺寸的截圖都沒有橫向溢出
- [ ] 所有圖片都載入成功
- [ ] 沒有 console 錯誤
- [ ] Claude 在建立 GitHub 專案之前，先把要執行的指令列給我看過
- [ ] 用瀏覽器打開 Claude 給的 GitHub 網址，看得到 `src` 資料夾和 `package.json`，沒有 `node_modules`

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
Vercel 部署失敗	打包在 Vercel 上出錯	把 Vercel 的完整錯誤訊息貼給 Claude，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」
網站打開一片空白	輸出目錄設定不對	「Vercel 上的輸出目錄應該設什麼？請告訴我要改哪一個欄位」
圖片沒出來	圖片連結失效	「請檢查所有圖片連結，把失效的列出來」

你可能會看到	通常是因為	怎麼辦
Vercel 的專案列表裡找不到剛建的專案	Vercel 還沒被允許讀取這個 GitHub 專案	把畫面截圖貼給 Claude：「Vercel 的列表裡找不到我剛建的 GitHub 專案，現在該點哪裡」
GitHub 推不上去	gh 還沒登入	回〈開始之前〉的〈接上 GitHub〉做一次。做完再回它：「可以，照你剛剛列的做。建好之後告訴我 GitHub 專案的網址。」

這一步失敗可以跳過嗎：可以。網站在你電腦上跑得起來，這個案例的學習目標就達成了。上線可以之後再補。

完成清單

做完之後逐項確認。**有一項打不了勾，就回去看對應的階段。**

- [ ] 我完成了課前作業，手上有一份自己寫的 `spec.md`
- [ ] 十二個階段的 ✅ 驗收都通過了
- [ ] `npm run build` 成功，型別檢查沒有錯誤
- [ ] 我在四個尺寸（1600 / 1280 / 834 / 390）都實際看過畫面
- [ ] 我用手機打開過（不是用電腦模擬）
- [ ] 我知道目前的成果是 **Functional Prototype**。我也說得出要升級還差哪三件事
- [ ] 我至少拒絕過一次 AI 的建議，而且說得出理由
- [ ] 我至少有一次因為「找不到對的素材」而選擇留白
- [ ] 我說得出這個案例的核心：需求、限制、驗收標準寫得越具體，Claude 要自己決定的部分越少

離開這個範例，我會不會自己做？

這幾題答不出來，代表那一段還沒學會。 前六題是回顧，後五題是遷移。

回顧

#	問題	對應階段
1	你的 `CLAUDE.md` 裡，哪一條規則是「可以被搜尋檢查的」？舉一個例子	5
2	為什麼自動檢查擋得住的東西，`CLAUDE.md` 擋不住？	8
3	你的網站拿掉 logo，還看得出是什麼行業嗎？	4、9
4	把照片換成咖啡店，你的版型還完全成立嗎？如果成立，問題出在哪？	4、9
5	你有沒有拒絕過 `/code-review` 的建議？為什麼？	11
6	你的強調色佔畫面幾 %？它的使用條件是什麼？	5

遷移

#	問題
7	如果今天不是陶藝工作室，而是一家健身房，你會先向業主問哪五種資訊？
8	你要幫一個朋友的甜點店做網站，他傳給你三張很喜歡的參考圖。你的第一個動作是什麼？
9	你要怎麼把「這個網站要有溫度」翻成三條機器檢查得了的規則？
10	你的下一個專案裡，哪些操作你會設定成「AI 不能自己做」？為什麼？
11	如果只能保留這個案例的一個習慣，你會留哪一個？

第 9 題答不出來的話，回去看階段 5。

常見卡關

症狀	原因	解法
`command not found: npm`	沒裝 Node.js	到 nodejs.org 下載 LTS 版本，一路按下一步。裝完把 Claude Code Desktop 關掉重開
Node 版本太舊（18、20）	Astro 需要 22.12 以上	重裝一次 LTS 版本
Windows 上 Code 分頁打不開	少了 Git for Windows	到 git-scm.com 安裝，裝完重開應用程式
`EACCES` / `permission denied`	那個資料夾不給寫	換一個資料夾，例如桌面上新建的
Views 選單裡沒有 Browser 面板	應用程式版本太舊	Claude → Check for Updates
自動檢查完全沒反應	Claude Code 只監看對話開始時就存在的資料夾	打一次 `/hooks` 或重開對話
`npm create astro` 出現 `Failed to add integrations`	指令裡有 `--add none`	請 Claude 拿掉這個參數
改了 `CLAUDE.md` 但沒生效	它是指向 `AGENTS.md` 的捷徑，被某些編輯器改成一般檔案	見階段 5 的 🆘
Claude 突然沒反應了	網站預覽用了前景模式，把它卡住	跟它說「請用背景模式啟動」
圖片沒有切出視窗邊緣	三層 CSS 互相影響	把現象描述給 Claude：「這張圖應該要切出視窗右緣，但它停在容器邊界」
中文標題被壓扁	用了 `transform: scaleX()`	「請改用 clamp() 控制字級，不要用變形」
生成的設計還是有 AI 感	規範寫成形容詞了	回到階段 5，把每一條改寫成可檢查的規則
卡超過 20 分鐘	原因不明	整段錯誤訊息貼給 Claude，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」還是不行就先跳過，每個階段的 🆘 都寫了能不能跳過

名詞白話對照表

這份表的解釋只寫到你做這個案例需要的程度。 建議印出來放在旁邊。

Claude Code 與工作流程

名詞	需要知道到什麼程度
Plan 模式	只規劃、不動手。在輸入框下方的模式選單切換。這個模式下 Claude 不會改任何檔案
Manual 模式	每一個改動都先跳卡片問你
Project folder	你指定給 Claude 工作的資料夾。它只會在這裡面動東西
CLAUDE.md	放在專案裡的一份說明書。每次新對話 Claude 都會先讀它，所以規則寫在這裡才會持續生效
AGENTS.md	跨工具的通用版說明書。Cursor、Copilot 也讀它。Astro 會把 `CLAUDE.md` 做成指向它的捷徑
捷徑（symlink）	檔案的捷徑。打開它其實是打開它指向的那個檔案
`/init`	內建指令，掃描專案後自動更新 `CLAUDE.md`
Skill	一包預先寫好的專業指示，用 `/名字` 叫出來
hook（自動檢查）	在 Claude 做某個動作之後自動執行的檢查。`CLAUDE.md` 是「請你這樣做」，hook 是「做完會被檢查」
結束代碼	腳本跑完回傳的數字。0 代表通過，非 0 代表失敗。自動檢查靠這個判斷要不要擋
commit	版本控制的一次存檔紀錄
git / 版本控制	記錄「什麼時候改了什麼」的系統。改壞了可以退回去
vibe coding	用自然語言帶著 AI 一起做東西的工作方式

網站與設計

名詞	需要知道到什麼程度
Astro	做網站的工具。網站拆成一塊一塊的檔案來寫，寫完再合成一般的網頁
TypeScript	JavaScript 的一種版本，可以在網站跑起來之前先抓出一部分錯誤。這個案例不用自己寫，只要知道 `astro check` 會用它檢查

名詞	需要知道到什麼程度
scaffold（專案產生器）	官方工具，一次把資料夾結構和設定檔建好
build（打包）	把你的原始檔案組裝成瀏覽器看得懂的成品，放在 `dist` 資料夾
`.js` / `.mjs`	都是 JavaScript 程式檔。`.mjs` 只是明確告訴電腦「這個檔案用比較新的寫法」。你不用分辨，AI 會挑對的那個
`.astro`	Astro 的頁面檔。一個檔案裡同時放這一頁的結構和樣式
`.gitignore`	一份清單，寫在裡面的檔案不會被版本控制記錄。例如 `node_modules/` 和放金鑰的設定檔
integration（擴充套件）	Astro 的外掛，例如加上 React 或 Tailwind 的支援。這個案例一個都不加
Design Token	把顏色、間距、圓角等數值集中定義成變數，全站引用變數而不是寫死數字
`clamp()`	CSS 函式，`clamp(最小, 想要的, 最大)`。用它做響應式字級，字不會被壓扁
lint（檢查）	自動檢查有沒有違反規則的工具。這個案例會請 Claude 寫一個檢查設計規範的
響應式（RWD）	同一個網站在不同螢幕寬度都能好好顯示
無障礙	讓視障、行動不便、只能用鍵盤的人也能使用
LCP	使用者多久看得到主要內容，越短越好
子集化	只從字體檔裡取出你真正用到的那幾個字。中文字體動輒好幾 MB，只要一個「器」字就沒必要全載
bleed（出血）	印刷用語，指圖片切出版面邊緣
eyebrow（小標）	標題上方那一行小字。模板網站每一段都放一個，所以它是 AI 感的來源之一
marquee（跑馬燈）	橫向不停捲動的文字帶
KPI row	三到四欄的大數字展示區（例如「10 年經驗 / 500 位客戶」）。模板公式之一
chips（標籤膠囊）	小小的圓角標籤。模板網站很愛用
backdrop-filter	讓元素背後的東西模糊，也就是毛玻璃效果
Vercel	一個放網站的服務，免費方案就夠這個案例用。跟 GitHub 連起來之後會自動更新
靜態網站	事先做好的一堆檔案，沒有伺服器在跑。便宜、快、不容易掛

各階段預估時間

階段	主題	讀完	動手
	開始之前：環境與四張操作卡	20 分	30 分
1	先看成品，找出「AI 感」	5 分	25 分
2	先讓 AI 提計畫，你確認後才動手	6 分	15 分
3	建立專案	7 分	20 分
4	把參考圖變成視覺規則	10 分	30 分
5	把視覺規則寫成設計規範	10 分	25 分
6	把顏色與尺寸集中成一份設定	6 分	15 分
7	做出第一個畫面	8 分	30 分
8	加上自動檢查	10 分	30 分
9	完成其餘區塊	8 分	40 分
10	量四個尺寸找出看不出來的問題	8 分	30 分
11	讓 AI 審查 AI，並學會拒絕	7 分	20 分
12	推上 GitHub 並部署	8 分	30 分
	合計	**約 1.9 小時**	**約 5.7 小時**

時間有限的話，可以先跳過 10–2 和階段 12，之後再回來補。 **不要跳過階段 8**，階段 5 寫的規則夠不夠具體，要到那裡才會被驗證。

建議分三次做：

分段	階段	產出
第一次	開始之前 → 階段 6	規則、規範、tokens 都定好了，但還沒有畫面
第二次	階段 7 → 階段 9	完整的首頁，而且受自動檢查保護
第三次	階段 10 → 階段 12	體檢、審查、上線

每一段結束時，談好的規則都已經寫進 `CLAUDE.md` 和檔案。隔幾天開新對話也接得上。